SC4063 Network Security

NTU BSc Computer Science — Comprehensive Textbook (0–100)

NTU CS Curriculum Textbook Series
College of Computing and Data Science

2026-08-28

Contents

Chapter 1

Threat Landscape and Attack Surface

> *"You cannot defend what you cannot see."* — The first job of network security is to draw an honest map: who wants in, where the doors are, and what lies behind each one.

From zero: no security background assumed. We start from a house with doors and valuables, turn it into packets on a wire, and build the vocabulary of assets, adversaries, and attack surfaces from scratch. Every term is defined on first use, picture first, then formalism. This chapter is the lens for the whole book.

Learning Outcomes

After this chapter you will be able to:

(L1) model the network adversary (Dolev–Yao, insider, APT) and its capabilities;

(L2) define *attack surface* and enumerate it from a network diagram;

(L3) map STRIDE/CIA threats to OSI/TCP-IP layers;

(L4) apply defence-in-depth, least privilege, and zero-trust reasoning;

(L5) prioritise controls with qualitative risk and CVSS intuition.

1.1 Who Is the Adversary?

Intuition first. A house assumes burglars exist; a network must assume the *road itself* is hostile. Packets travel over fibre, Wi-Fi, and routers you do not control. The strongest useful abstraction is the *Dolev–Yao* attacker: she can *eavesdrop* every packet, *modify* or *inject* any packet, *replay* old packets, and *drop* packets at will, but cannot break cryptography without the key.

Definition 1.1 (Network adversary)**.** The *Dolev–Yao* adversary controls the network: read, modify, inject, replay, and drop. An *insider* is an authorised user who abuses privilege. An *advanced persistent threat* (APT)

is a well-resourced actor who combines network, host, and social vectors over months. All three are assumed to be active simultaneously.

Why assume such power? Because the Internet delivers it cheaply: a café Wi-Fi lets anyone sniff, a compromised router lets anyone modify, and botnets let anyone flood. Designing for a weaker attacker yields a system that fails the day someone stronger appears. Cryptography is the only edge case where the adversary is bounded: without the key, a ciphertext looks random.

Threat actors differ in motive, resources, and stealth. A script kiddie runs ready-made exploits for fun; a criminal syndicate monetises data; a nation-state seeks persistence and deniability. The network sees the same SYN scan from all three, but the follow-up differs: the first stops, the second deploys ransomware, the third implants a backdoor that beacons slowly.

Example 1.1 (Coffee-shop attack)**.** Alice joins `Free_WiFi`, opens `http://intranet.corp/`. The attacker runs `arpspoof` to become the gateway, strips the redirect to HTTPS, and sees Alice's cookie. Dolev–Yao predicted every step: eavesdrop, inject (ARP), modify (strip), replay (cookie). TLS (Ch. 4/8) would have made the ciphertext useless, but only if Alice had verified the certificate.

1.2 Attack Surface: Where Can They Enter?

An *asset* is what you protect (data, service, reputation); an *attack surface* is the set of points where untrusted data can enter or trusted data can leave. Smaller surface means fewer places to be wrong.

Definition 1.2 (Attack surface)**.** The *attack surface* of a networked system is the set $\mathcal{S} = (E, F, T)$ of *entry points* E (open ports, APIs, wireless SSIDs), *flows* F (packet paths between trust zones), and *trust boundaries* T where privilege changes. Each element of $\mathcal{S}$ is an opportunity to apply STRIDE analysis.

Draw it. Start with a data-flow diagram: external entities (Internet, partner), processes (web server, DB), stores (DB, object storage), and flows (arrows). Dashed lines mark trust boundaries; every crossing is a candidate entry. Then count: each open TCP port, each UDP service, each SSID, each VPN endpoint adds one entry. The perimeter is not one line but many: edge router, WAF, Wi-Fi, and every laptop that leaves the building.

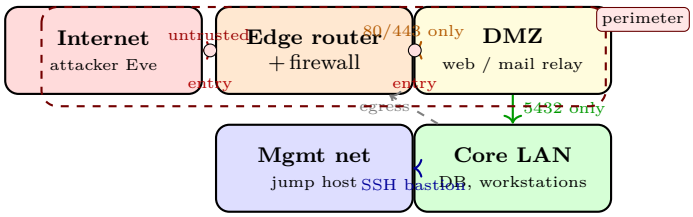


Figure 1.1: Network perimeter with trust boundaries. Red dots are entry points where attacker-controlled packets cross inward. Every crossing needs a threat class and a control.

Reducing surface is engineering: close unused ports, put the DB on a private subnet with no Internet route, enforce egress filtering (compromised hosts should not call home on arbitrary ports), and treat every laptop's Wi-Fi as untrusted even inside the office. A VPN (Ch. 4) does not shrink the surface; it moves it to the VPN gateway, which must then be hardened.

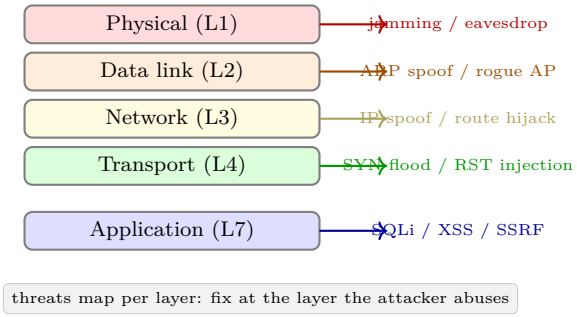


Figure 1.2: OSI/TCP-IP layer vs. threat badge. Lower layers attack availability and authenticity; upper layers attack confidentiality and integrity.

Listing 1.1: Enumerating the network attack surface from a simple port scan.

```
import socket
TARGETS = ["10.0.0.5","10.0.0.12","10.0.2.8"]
PORTS   = [22,80,443,5432,6379]  # candidates to probe
surface = []
for host in TARGETS:
    for port in PORTS:
        s = socket.socket(); s.settimeout(0.4)
        if s.connect_ex((host, port)) == 0:  # open
            surface.append((host, port))
        s.close()
print(f"attack surface: {surface}")
# -> [('10.0.0.5', 22), ('10.0.0.5', 443), ('10.0.2.8', 5432)]
# Each tuple is an entry point that needs a STRIDE row.
```

1.3 Threat Classification and Risk

STRIDE partitions threats by the property violated: Spoofing (authenticity), Tampering (integrity), Repudiation, Information disclosure (confidentiality), Denial of service (availability), Elevation of privilege (authorisation). Apply it per flow: a packet crossing the Internet→DMZ invites S/T/I/D; a DB query from DMZ→core invites T/I/E.

Risk prioritises the list. Qualitatively, Risk = Likelihood × Impact on a 3×3 matrix; quantitatively, CVSS v3 scores 0–10 from exploitability and impact metrics. A reflected XSS on an internal tool may be medium; unauthenticated RCE on the DMZ web server is critical (CVSS 9.8) and fixed before release.

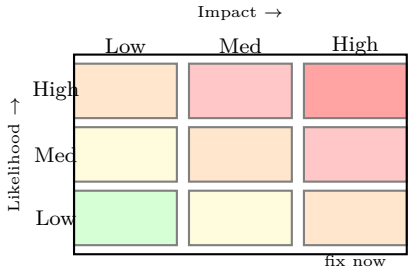


Figure 1.3: Risk heatmap (likelihood × impact). Colour deepens toward critical. CVSS refines each axis into exploitability and impact sub-scores.

Two principles guide controls. *Defence in depth* means an attacker must defeat several independent controls (firewall + IDS + TLS + auth). *Least privilege* means each flow is allowed the narrowest 5-tuple that still serves its purpose. Zero trust pushes both to the limit: never trust the network, always authenticate and authorise per flow. The perimeter becomes a set of micro-perimeters around every service.

Remark 1.1 (Threat modelling is iterative). Every feature adds flows and stores. "Add OAuth" adds a redirect flow that crosses the trust boundary twice; "allow file upload" adds storage and a rendering flow in another user's browser. Re-run STRIDE on the delta; if the cheapest attack path after the fix is not strictly more expensive, the fix was cosmetic. Models that do not evolve become fiction.

1.4 Exercises

E1.1 **Dolev–Yao trace.** Alice sends m over Wi-Fi to Bob. List five distinct actions a Dolev–Yao attacker can take on that single transmission and name the CIA property each violates.

E1.2 **Attack surface enumeration.** Draw a DFD for a campus app: mobile→API gateway→app service→DB, plus push notifications to recipients. Mark trust boundaries, list every entry point, and identify the entry with highest asset exposure. Propose one reduction.

E1.3 **STRIDE per layer.** For the flow `client`→`CDN`→`origin`→`DB`, enumerate which STRIDE letters apply at each hop. Give one concrete exploit per letter that actually applies; omit letters that do not.

E1.4 **Risk scoring.** Score two threats with CVSS intuition: (a) reflected XSS on search and (b) unauthenticated DB backup at `/backup.zip`. Assign exploitability and impact (low/med/high), estimate a 0–10 score, and rank. Which mitigation ships first and why?

E1.5 **Zero trust vs. perimeter.** A laptop on the office LAN is compromised. Explain why a perimeter model fails to contain it while a zero-trust model limits blast radius. Name three concrete micro-perimeter controls that would have helped.

Chapter Notes

Dolev–Yao (1983) formalises the network attacker; STRIDE is Microsoft (1999; Shostack 2014). Attack-surface formalisation follows Manadhata–Wing (2011). CVSS v3.1 (FIRST, 2019) and NIST SP 800-154 (threat modelling) are practical references. Defence-in-depth and zero trust are codified in NIST SP 800-207 (2020) and the NSA Cybersecurity Advisory on network segmentation.

Chapter 2

Firewalls and Network Segmentation

> *"A firewall is a policy made of packets."* — If the network is a city, the firewall is the checkpoint that asks every car for papers and decides, by rule, who passes and who is turned away.

From zero: packets first. We start from a single IP header, add one rule, then a table, then state, then an application parser. Every concept is built incrementally: intuition (checkpoint) → header fields → rule → connection table → segmented topology. No prior firewall knowledge assumed.

Learning Outcomes

After this chapter you will be able to:

(L1) distinguish packet-filter, stateful, proxy, and NGFW generations;

(L2) write and audit an `iptables`/`nftables` rule set and explain first-match semantics;

(L3) explain conntrack and the TCP state machine for stateful filtering;

(L4) design a screened-subnet DMZ and reason about micro-segmentation;

(L5) evaluate firewall limits (encryption, insider, covert channels) and pair with IDS.

2.1 What a Firewall Sees

A flow is the 5-tuple (srcIP, dstIP, srcPort, dstPort, proto). A firewall policy is a function f: 5-tuple → {allow, deny, log}. The wire does not provide intent, only headers and perhaps a payload. Each generation of firewall sees strictly more context at higher cost.

Packet filter (stateless) Matches header fields only. Rule `deny tcp any any eq 23` blocks Telnet anywhere. Fast, cheap, but cannot tell whether a packet belongs to an existing flow; return traffic needs a broad reverse rule that attackers abuse.

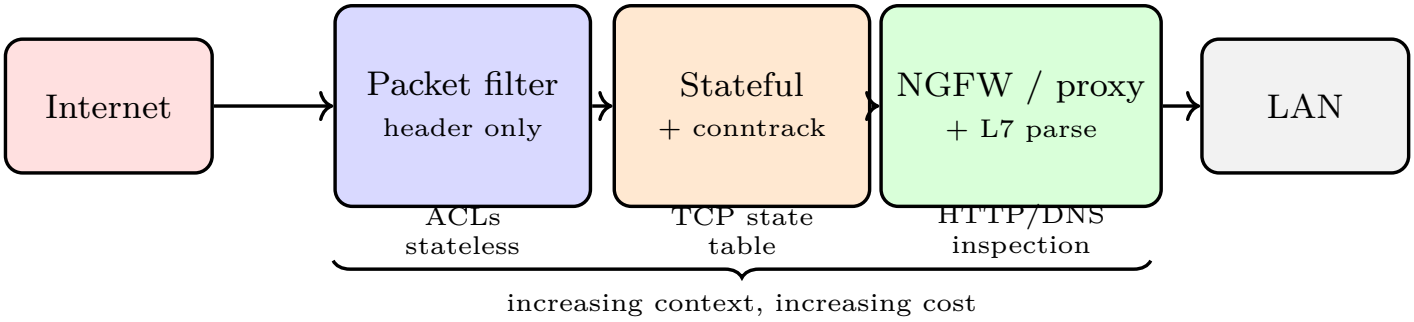


Figure 2.1: Firewall generations as a pipeline. Packet filters match headers; stateful adds a connection table; NGFW/proxy terminates and parses the application.

Stateful firewall Keeps a *state table* of active flows (SYN seen, ESTABLISHED, RELATED). Return SYN-ACK is allowed only if a matching SYN is in the table, blocking unsolicited inbound TCP and spoofed ACKs. This is `conntrack` in Linux.

Application proxy / NGFW Terminates the connection, parses the application (HTTP, DNS, TLS SNI), and enforces content policy: `deny http POST to /admin from !corp`, strip ActiveX, block DNS tunnelling. Highest visibility, highest latency and certificate cost. A WAF is this idea specialised to HTTP.

Example 2.1 (Rule order matters)**.** Default-deny with rules in order: (1) `allow tcp 10.0.0.0/8 any eq 443`, (2) `allow tcp any 10.0.0.5 eq 80`, (3) `deny ip any any`. A packet to `10.0.0.5:80` matches (2); placed below (3) it would be dropped. First-match semantics and shadowing cause most firewall outages. `iptables -L -n -line-numbers` audits order.

2.2 Stateful Filtering and Conntrack

Stateless "allow `any any established`" lets an attacker reach an internal service by setting the ACK bit. Stateful fixes it: only packets belonging to a known connection pass.

Definition 2.1 (State table entry)**.** An entry keys on the 5-tuple plus TCP state {NEW, ESTABLISHED, RELATED, INVALID}, timeout, and counters. On outbound SYN, insert NEW; on matching SYN-ACK, promote to ESTABLISHED; on FIN/RST, schedule expiry. The reverse direction inherits permission from the forward entry.

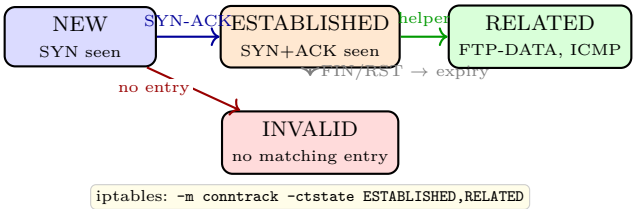


Figure 2.2: Conntrack state machine (simplified). Only ESTABLISHED/RELATED return traffic is allowed, blocking spoofed ACKs that stateless filters admit.

Linux implements this as `nf_conntrack`. Every NAT translation also consumes a conntrack entry, so table exhaustion is a DoS vector (Ch. 5).

Listing 2.1: Stateful iptables: default-deny with conntrack.

```
# Default deny
iptables -P INPUT DROP
iptables -P FORWARD DROP
iptables -P OUTPUT DROP
# Allow loopback and established return traffic
iptables -A INPUT  -i lo -j ACCEPT
iptables -A INPUT  -m conntrack --ctstate ESTABLISHED,RELATED -j ACCEPT
# Allow outbound 80/443 and DNS to 8.8.8.8, log the rest
iptables -A OUTPUT -o eth0 -p tcp --dport 80  -m conntrack --ctstate NEW -j ACCEPT
iptables -A OUTPUT -o eth0 -p tcp --dport 443 -m conntrack --ctstate NEW -j ACCEPT
iptables -A OUTPUT -o eth0 -p udp --dport 53 -d 8.8.8.8 -j ACCEPT
iptables -A OUTPUT -j LOG --log-prefix "DROP-OUT: "
# Shadowed-rule audit: insert a broad deny before a narrow allow and watch it break
```

`nftables` replaces the four tables with one rule set and supports sets and intervals, but the semantics (first match, conntrack) are identical.

2.3 Segmentation: DMZ and Beyond

One firewall is one failure. A *screened subnet* places public servers in a DMZ between two firewalls: outer allows Internet→DMZ on 80/443, inner allows DMZ→LAN only for needed flows (app→DB on 5432). Compromise of the DMZ does not directly expose the LAN.

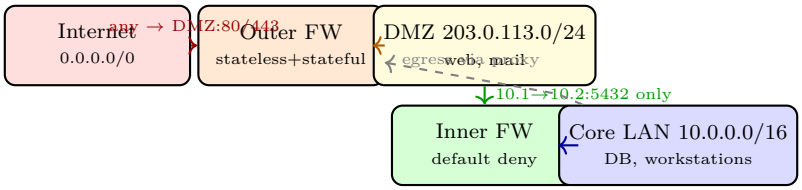


Figure 2.3: Screened-subnet DMZ. Two firewalls enforce least privilege per hop. Egress from LAN is proxied and logged, not routed directly.

Modern networks push further: *micro-segmentation* treats every workload as its own zone (Kubernetes NetworkPolicy, host firewalls, identity-aware proxies). NAT hides private addresses but is not a security control; a misconfigured NAT that hairpins or forwards ports reopens the surface.

Listing 2.2: Toy stateless firewall: first-match ACL evaluation.

```
rules = [("allow","10.0.0.0/8","any",443), ("allow","any","10.0.0.5",80),
         ("deny","any","any","any")]  # order matters
import ipaddress
def match(pkt, rule):
    _, src, dst, dport = rule
    def net_match(net, ip):
        return net=="any" or ipaddress.ip_address(ip) in ipaddress.ip_network(net)
    return net_match(src, pkt["src"]) and net_match(dst, pkt["dst"]) and (dport=="any" or
        pkt["dport"]==dport)

pkt = {"src":"8.8.8.8","dst":"10.0.0.5","dport":80}
for r in rules:
```

```
    if match(pkt, r):
        print(r[0], "by", r); break  # first match wins
```

Firewalls are blind to encrypted payloads, insider traffic, and covert channels (DNS tunnelling, ICMP exfil). That is why every firewall deployment is paired with IDS/IPS (Ch. 3) on a decrypted copy and with host controls. A firewall that logs nothing is a firewall that cannot be audited.

Remark 2.1 (Egress matters more). Ingress rules block outsiders; egress rules catch insiders and compromised hosts calling home. An allow-list for outbound (only proxy → 80/443, DNS to vetted resolvers) stops most command-and-control without touching a single ingress rule.

2.4 Exercises

E2.1 **ACL audit.** Write a default-deny rule set for `203.0.113.10` that allows inbound 443/80 from anywhere, outbound DNS (UDP 53) to `8.8.8.8`, and blocks Telnet (23) explicitly. Identify one shadowed rule if the deny-all is moved to the top.

E2.2 **Stateless vs. stateful.** A stateless filter has `allow tcp any any established`. Show how an attacker reaches an internal service on 8080 by setting ACK. What exact conntrack entry after a legitimate SYN would a stateful firewall check to block that attack?

E2.3 **DMZ design.** Place a 3-tier app (WAF, app servers, DB) in a screened subnet. For each firewall, list allowed 5-tuples. Explain why the DB must have no route to the Internet even via NAT.

E2.4 **iptables lab.** Translate the rule set in Listing 2.1 to `nftables` syntax. Add a rate-limit of 20 new SSH connections per minute with `-limit` and explain where it mitigates brute force.

E2.5 **Limits of firewalls.** Give one attack that a correctly configured NGFW still misses for each of: (a) encrypted payload, (b) insider, (c) DNS covert channel. For each, name the complementary control (IDS, DLP, DNS filter).

Chapter Notes

Firewalls originate from Cheswick–Bellovin (1994); stateless vs. stateful is codified in Chapman–Zwicky (1995). Linux conntrack and `iptables`/ `nftables` are documented in `netfilter.org` and Michael Rash's `Netfilter` references. NGFW/WAF trade-offs follow NSS Labs tests and NIST SP 800-41 Rev 1. DMZ and segmentation patterns are in NIST SP 800-41 and NSA's network segmentation guidance; micro-segmentation aligns with NIST SP 800-207 (Zero Trust).

Chapter 3

Intrusion Detection and Prevention

> *"Firewalls decide who may pass; IDS decides who already passed and looks guilty."* — One enforces a guest list, the other watches the crowd.

From zero: seeing is a system. We start from a single packet copy, ask what to compare it against (a signature or a model of normal), then build the pipeline that turns copies into alerts and, optionally, into drops. No IDS background assumed; every component is motivated before it is named.

Learning Outcomes

After this chapter you will be able to:

(L1) contrast NIDS, NIPS, and HIDS placement and fail-open vs. fail-closed;

(L2) distinguish signature, anomaly, and hybrid detection and their ROC trade-offs;

(L3) author and tune Snort/Suricata rules and suppress false positives;

(L4) explain base-rate fallacy, evasion, and encrypted-traffic limits;

(L5) correlate alerts via SIEM and argue IDS vs. IPS blocking policy.

3.1 Where the Sensor Sits

A firewall is inline by definition; an IDS may be passive (copy) or inline (block). The vantage determines risk.

Definition 3.1 (NIDS vs. NIPS vs. HIDS)**.** A *NIDS* copies traffic via SPAN/mirror or TAP, raises alerts, never blocks. A *NIPS* is inline and can drop, reset, or quarantine; every NIPS is a NIDS that also enforces. A *HIDS* runs on the host and sees syscalls, logs, and file integrity. Network sees the wire; host sees the outcome.

Inline placement creates a failure-mode choice. *Fail-open*: if the NIPS crashes, traffic passes uninspected (availability over security). *Fail- closed*: traffic stops (security over availability). Data centres choose fail- open

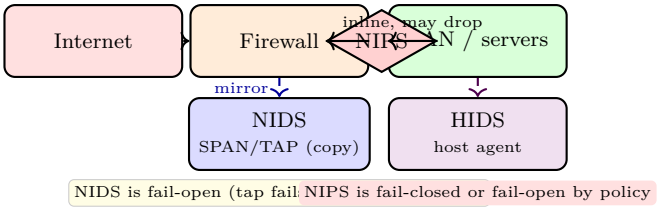


Figure 3.1: Sensor placement. NIDS taps a copy; NIPS sits inline as a gate; HIDS watches the host. Colour distinguishes passive (blue/violet) from enforcing (red/orange).

for revenue; safety-critical networks choose fail-closed.

Modern EDR/XDR fuses all three: network metadata plus host telemetry in one timeline. Encrypted traffic blinds the network sensor, pushing visibility to the endpoint or to a TLS-intercept proxy with explicit consent.

3.2 What to Compare Against

Two philosophies compete: look for the known bad, or model the known good.

Signature-based Matches known patterns: byte sequences, regex on headers/payload, or Snort rules. `alert tcp any any -> any 80 (msg:"SQLi"; content:"' OR 1=1";)` has low false positives but is blind to zero-days and needs frequent updates. Analogous to antivirus signatures. Snort (single-threaded) and Suricata (multi-threaded, YAML config) dominate open source.

Anomaly-based Learns a model of normal (traffic volume, n-gram entropy, host behaviour) and flags deviation beyond a threshold. Catches novel attacks; suffers higher false positives and adversarial mimicry (attacker shapes traffic to look normal). Needs clean training data.

Hybrid Signature for the known, anomaly for the unknown, with a correlation layer (SIEM) to combine. Production default.

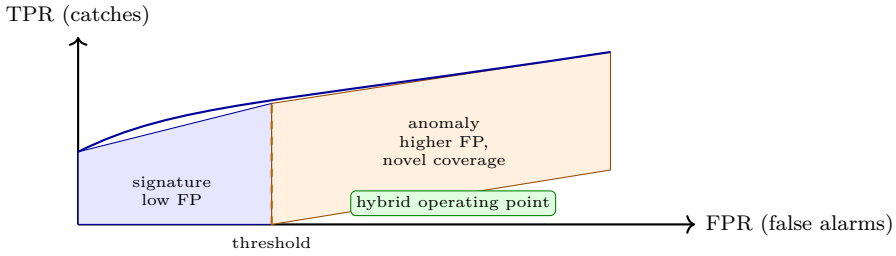


Figure 3.2: ROC intuition. Signatures sit at low FPR with a ceiling; anomaly pushes the curve outward at higher FPR. Operators choose a threshold per response (block vs. alert).

Metrics: true/false positives/negatives, ROC curve, and the *base-rate fallacy*: even 1% FPR on 1 M daily events with 10 true attacks yields ≈10 000 alerts and precision 0.1%. That is why signatures dominate inline blocking (low FP required) while anomaly is often alert-only for a SOC analyst.

Listing 3.1: Toy IDS: signature match + entropy anomaly sketch.

```
import re, math, collections
RULES = [re.compile(p) for p in [r"' OR 1=1", r"<script", r"\.\./"]]
def inspect(payload: str):
```

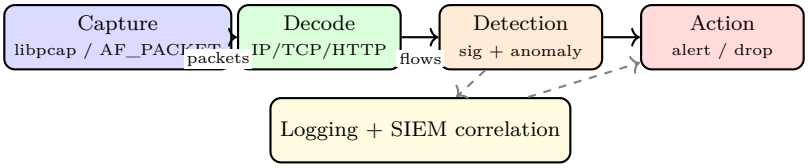


Figure 3.3: IDS pipeline. Every stage can be tuned: capture performance (AF_PACKET, DPDK), decode coverage, detection thresholds, and action policy.

```
    for r in RULES:
        if r.search(payload):
            return f"ALERT sig:{r.pattern!r}"
    return "OK"
# Anomaly: byte entropy deviates >2 sigma from baseline
baseline = (4.2, 0.6)  # mean, stdev of entropy for normal HTTP payloads
def entropy(s):
    from collections import Counter
    c = Counter(s.encode()); n = len(s) or 1
    return -sum((v/n)*math.log2(v/n) for v in c.values())
def anomaly(payload):
    e = entropy(payload)
    return abs(e-baseline[0]) > 2*baseline[1]
print(inspect("GET /?q=' OR 1=1 --"))  # ALERT sig
print(anomaly("A"*800 + "\x00\xff"*20))  # True if entropy odd
```

Listing 3.2: Suricata rule and tuning snippet.

```
# /etc/suricata/rules/local.rules  -- signature
alert http any any -> any any (msg:"SQLi UNION SELECT"; \
  http.uri; content:"UNION"; nocase; content:"SELECT"; distance:0; \
  classtype:web-application-attack; sid:1000001; rev:1;)
# suricata.yaml -- suppress a noisy false positive for a known scanner
# threshold: type suppress, track by_src, ip 10.0.2.15, sid 1000001
```

Tuning is the job: suppress noisy SIDs, add flowbits to correlate across packets, and write threshold rules (`5 hits in 10s`) to turn probes into incidents. Without tuning, an IDS becomes an alert firehose that hides real compromise.

Evasion reminds us detection is adversarial. Attackers fragment packets, vary case, encode payloads, or tunnel over encryption so the NIDS never sees the bytes the endpoint reassembles. Normalisation (reassemble before matching) and endpoint visibility are the counters; they cost CPU and are never perfect.

Remark 3.1 (Encrypted traffic). TLS blinds payload inspection. Options: (a) inspect at the endpoint (HIDS), (b) decrypt at a forward proxy with consent and re-encrypt (enterprise TLS inspection, privacy trade-off), or (c) detect on metadata (flow sizes, timing, JA3 fingerprints, DNS). Choice is policy, not just technology.

3.3 Exercises

E3.1 **Placement.** A 10 Gbps link carries trading traffic. Argue for NIDS vs. NIPS, fail-open vs. fail-closed, and where you would also need HIDS. What does encryption do to your choice?

E3.2 **ROC arithmetic.** A signature set has 1% FPR on 1 M daily events with 10 true attacks. Compute expected alerts and precision. An anomaly detector catches 9/10 at 5% FPR. Compute its alerts and say which is suitable for inline blocking vs. SOC monitoring.

E3.3 **Rule authoring.** Write a Suricata rule that alerts on `../../etc/passwd` in `http.uri` (case-insensitive) and add a `threshold` to alert only if seen 3 times from the same source in 60s. Explain how to suppress it for a scanner at `10.0.2.15`.

E3.4 **Base-rate intuition.** Even with 99% TPR and 1% FPR, why does a base rate of 1 attack in 100 000 flows make most alerts false? Compute posterior probability of a true attack given an alert (Bayes).

E3.5 **Evasion.** Show two distinct ways to evade a naive `content:"cmd.exe"` NIDS signature without changing the endpoint's behaviour (fragmentation, encoding, case, etc.). Propose a normalisation that defeats each.

Chapter Notes

IDS taxonomy is Denning (1987) and Axelsson (2000); Snort (Roesch, 1999) and Suricata (OISF, 2010) exemplify signatures. Sommer–Paxson (2010) critiques anomaly hype; fraud-base-rate analysis follows Axelsson. Evasion via insertion and fragmentation is Ptacek–Newsham (1998). SIEM correlation and encrypted- traffic visibility are covered in NIST SP 800-94 (IDS/IPS) and SP 800-83 (malware/EDR). Zeek (formerly Bro) is the canonical network monitor for metadata-first detection.

Chapter 4

VPNs and Secure Tunnelling

> *"A VPN is a private road painted on a public map."* — Packets still cross the Internet, but observers see only the outer envelope; the inner letter is sealed and addressed to a private destination.

From zero: envelopes inside envelopes. We start from a plain IP packet, wrap it in a new envelope (tunnel), seal it with authenticated encryption, and negotiate the sealing key. Intuition (postal tunnel) → encapsulation → key exchange → live config. No prior VPN knowledge assumed.

Learning Outcomes

After this chapter you will be able to:

(L1) explain tunnel vs. transport encapsulation and the need for a security association;

(L2) describe IPsec ESP/AH, IKEv2, and the TLS VPN alternative;

(L3) describe WireGuard's Noise handshake and why it is simpler;

(L4) choose split vs. full tunnel, remote-access vs. site-to-site, and kill-switch policy;

(L5) audit a VPN for DNS, IPv6, and reconnection leakage and reason about overhead.

4.1 Why Tunnel at All?

A private network (10.0.0.0/8) is not routable on the public Internet. A *VPN* creates a virtual point-to-point link over that public network by *encapsulating* and *encrypting*. Routing decides what enters the tunnel; cryptography ensures outsiders see only ciphertext and the outer headers. Think of putting the entire letter, with its original envelope, inside a new outer envelope addressed gateway-to-gateway.

Definition 4.1 (Security association)**.** A *security association* (SA) is simplex state for one direction: encryption/integrity algorithms, keys, sequence counter, anti-replay window, and lifetime. IPsec needs two SAs per flow (inbound/outbound); rekeying before expiry preserves forward secrecy of the data channel.

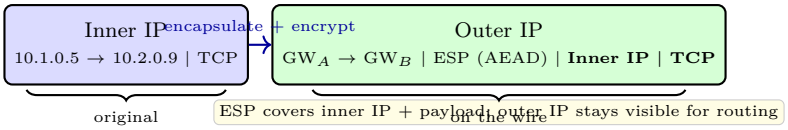


Figure 4.1: IPsec ESP in tunnel mode. The entire inner packet is encrypted and authenticated; a new outer IP header carries it between gateways. Transport mode would encrypt only the payload and keep the original IP header.

Two modes cover two needs. *Transport* encrypts the payload and keeps the original IP header: host-to-host without a gateway. *Tunnel* encrypts the whole IP packet and adds a new outer header: gateway-to-gateway or remote-access. On the wire, `[Inner IP|TCP]` becomes `[Outer IP|ESP|Inner IP|TCP]`; AH (integrity only, now rare) is similar but without confidentiality.

4.2 IPsec, TLS VPN, and WireGuard

IPsec is the network-layer suite: ESP for confidentiality+integrity and IKEv2 for key management. *TLS VPN* runs over TCP 443 and looks like HTTPS to middleboxes. *WireGuard* is the modern minimalist alternative.

IPsec + IKEv2 Kernel-level, one SA per flow, strong for site-to-site (always on, transparent to apps). NAT traversal needs NAT-T (UDP 4500) and is brittle. Key exchange is IKEv2 (Diffie–Hellman + authentication), rekeying without dropping packets.

TLS VPN User-space, firewall-friendly (TCP 443), per-application granularity. Two flavours: *portal* (reverse-proxy to web apps) and *tunnel* (TUN/TAP adapter carrying IP). Preferred for BYOD road warriors through hotel NATs; easy to deploy, heavier per-packet overhead.

WireGuard One RTT handshake (Noise_IK), static + ephemeral Curve25519, ChaCha20-Poly1305, BLAKE2s, short codebase (∼4 k lines). Roaming-friendly (IP is not identity; cryptokey routing), built into the Linux kernel, no cipher agility, excellent for remote-access and site-to-site alike.

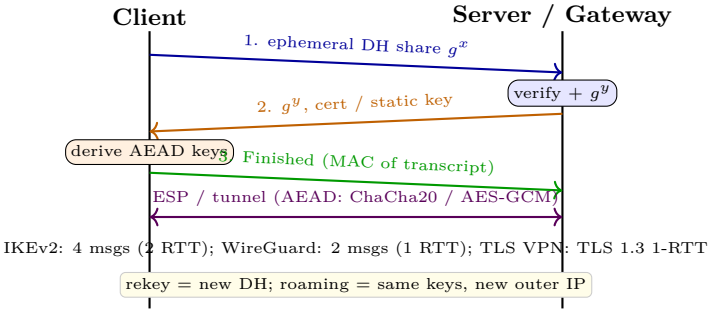


Figure 4.2: Handshake ladder (IKEv2 / WireGuard / TLS VPN, simplified). ephemeral DH gives forward secrecy; the Finished MAC binds the transcript and prevents downgrade.

Listing 4.1: WireGuard: minimal site-to-site and client config.

```
# /etc/wireguard/wg0.conf  -- each peer is one public key + allowed IPs
[Interface]
PrivateKey = <client-private>    # Curve25519
Address = 10.8.0.2/32
DNS = 10.8.0.1                   # push private DNS to avoid leak
```

	IPsec	TLS VPN	WireGuard
Layer	L3 kernel, SA per flow	L4/L7 user-space	L3 kernel, cryptokey routing
Through NAT	Hard (ESP, NAT-T)	Easy (TCP 443)	Easy (UDP, roaming)
Client	Heavy (strongSwan)	Light (browser/agent)	Very light (`wg`)
Crypto	Agility (many suites)	TLS 1.3 suites	Fixed (Noise, ChaCha20)

Table 4.1: Choosing a VPN. Site-to-site on stable links favours IPsec; BYOD road warriors favour TLS VPN or WireGuard; simplicity and roaming favour WireGuard.

```
[Peer]  # gateway
PublicKey = <gateway-public>
AllowedIPs = 10.0.0.0/16, 10.8.0.0/24  # cryptokey routing: what goes into tunnel
Endpoint = gw.corp.example:51820
PersistentKeepalive = 25        # keep NAT mapping alive
# up: wg-quick up wg0  |  down: wg-quick down wg0
```

WireGuard's elegance is cryptokey routing: "which packets go where" is not a routing table but "which public key is allowed for which IPs." No certificates, no IKE ciphers to negotiate, roaming is free because the endpoint IP is just a hint that updates on any authenticated packet.

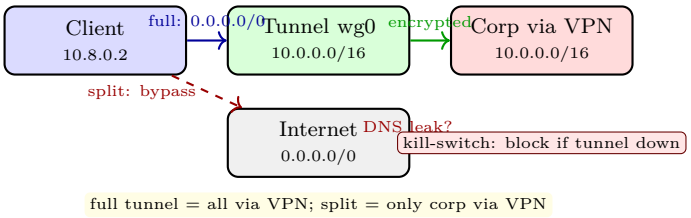


Figure 4.3: Split vs. full tunnel. Full tunnel protects all traffic but costs bandwidth; split saves bandwidth but must not leak DNS or IPv6 around the tunnel.

4.3 Leakage, Performance, and Policy

A VPN that leaks is worse than no VPN because it creates false confidence. Audit three leaks: *DNS* (resolver bypasses tunnel), *IPv6* (tunnel is v4-only, OS prefers v6), and *reconnection* (tunnel down, traffic falls back to clear). Mitigations: push a private DNS and block port 53 outside the tunnel, disable or tunnel IPv6, and enforce a *kill switch* (default-deny without the SA, Listing 4.2).

Listing 4.2: Kill-switch and leak audit (nftables + checks).

```
# nftables kill-switch: only allow out via wg0 or to the gateway
nft add rule inet filter output oifname != "wg0" ip daddr != 203.0.113.1 drop
# audit: DNS must not leave outside tunnel
tcpdump -i eth0 -n port 53 &  # should see nothing after VPN up
# audit: IPv6 must not bypass
ip -6 route | grep -v wg0 && echo "IPv6 leak!"
```

Performance: AEAD adds per-packet overhead (ESP header + tag ≈30–50 B) and crypto cost; WireGuard's ChaCha20 is fast without AES-NI, IPsec AES-GCM is fast with it. One SA per flow (IPsec) vs. one interface (WireGuard) vs. one TLS stream (TLS VPN) changes how head-of-line blocking and rekeying behave. Measure, do not guess: `iperf3` inside vs. outside the tunnel.

Remark 4.1 (VPN does not replace endpoint security). A compromised laptop inside the tunnel is still inside the perimeter. Combine VPN with device posture (patch level, EDR), MFA for the handshake, and short-lived credentials. Split tunnelling (only corporate traffic via VPN) trades bandwidth for exposure and must be policy-controlled, not user-chosen.

4.4 Exercises

E4.1 **Tunnel vs. transport.** Two gateways link `10.1.0.0/16` and `10.2.0.0/16` via IPsec ESP. Show the on-wire packet for `10.1.0.5`→`10.2.0.9` in tunnel and in transport mode. Which hides the inner addresses from an observer?

E4.2 **Protocol choice.** A company needs (i) permanent link between two data centres and (ii) remote access for 500 BYOD laptops through hotel NATs. For each, choose IPsec vs. TLS VPN vs. WireGuard and justify with NAT traversal, client provisioning, and granularity.

E4.3 **WireGuard config.** Given Listing 4.1, what happens if `AllowedIPs` mistakenly includes `0.0.0.0/0` on the gateway? What happens if `PersistentKeepalive` is omitted behind a NAT?

E4.4 **Leak audit.** Describe a step-by-step test that proves a VPN leaks DNS (record the `tcpdump` filter and expected output). Propose a fix via pushed DNS and a firewall rule that blocks port 53 outside the tunnel.

E4.5 **Performance.** ESP adds 38 B per packet (header+IV+tag). For 1400 B payloads, compute overhead %. An `iperf3` run shows 920 Mbps outside, 840 Mbps inside. Is the drop fully explained by overhead? What else could contribute (crypto, MTU, rekey)?

Chapter Notes

IPsec is RFC 4301/4303 (ESP) and RFC 7296 (IKEv2, Kaufman et al.); TLS VPN trade-offs follow NIST SP 800-77 Rev 1 (Frankel et al.). WireGuard is Donenfeld, "WireGuard: Next Generation Kernel Network Tunnel" (NDSS 2017) and `wireguard.com`. Comparison with TLS 1.3 handshake (Rescorla, RFC 8446) clarifies forward secrecy. Leakage and kill-switch guidance follows Mullvad and ANSSI hardening guides; performance numbers assume AES-GCM vs. ChaCha20 on modern x86/ARM with/without AES-NI.

Chapter 5

Denial-of-Service and Availability Defence

"Availability is the quiet property: you notice it only when it is gone." — Confidentiality and integrity hide secrets; availability keeps the door open for the honest while closing it for the flood.

From zero: flooding a pipe until the honest cannot speak. We start from one SYN, add ten thousand bots, then a reflector that shouts back louder than it was spoken to. Picture first (bot → amplifier → victim), then arithmetic (amplification factor), then defence (cookies, policing, scrubbing). No prior DoS background assumed.

Learning Outcomes

After this chapter you will be able to:

(L1) classify DoS/DDoS as volumetric, protocol, or application-layer and give an example of each;

(L2) explain botnets, reflectors, and pulsing attacks and compute an amplification factor;

(L3) describe the SYN-flood backlog and how SYN cookies trade state for computation;

(L4) compare rate policing, blackholing, scrubbing centres, and Anycast/CDN absorption;

(L5) execute a detect→classify→filter→scale→post-mortem playbook.

5.1 Taxonomy: What Does "Down" Mean?

Availability means an authorised request receives a correct response within its deadline. Denial-of-service breaks that promise by exhausting a bottleneck: bandwidth, packets-per-second, connection state, or application logic. One sender is DoS; many senders is DDoS and is strictly harder because the ingress is diffuse.

Volumetric Fill the pipe. UDP floods, DNS/NTP amplification, and carpet-bombing saturate links or PPS budgets. Defence is absorbing or diverting bits elsewhere (Anycast, CDN, scrubbing). Rate limits near the victim help little because the link before the filter is already full.

Protocol Abuse the handshake, not the payload. SYN flood leaves half-open state,fragment floods exhaust reassembly, and Slowloris holds application slots with partial headers. Defence is making state cheap or stateless (SYN cookies, timeouts) and policing per-source.

Application Look legitimate but cost dearly. HTTP GET floods, expensive API calls, and TLS renegotiation force the server to do work the attacker avoids. Defence needs application awareness: WAF rules, proof-of-work, and autoscaling with a cost ceiling.

Pulsing and carpet-bombing add time and space: short bursts evade static thresholds, and spreading traffic across many destination IPs in one prefix evades per-IP policing. A carpet-bomber that spreads 1 Tbps over a /24 forces the defender to filter a prefix, not a host, which raises collateral damage.

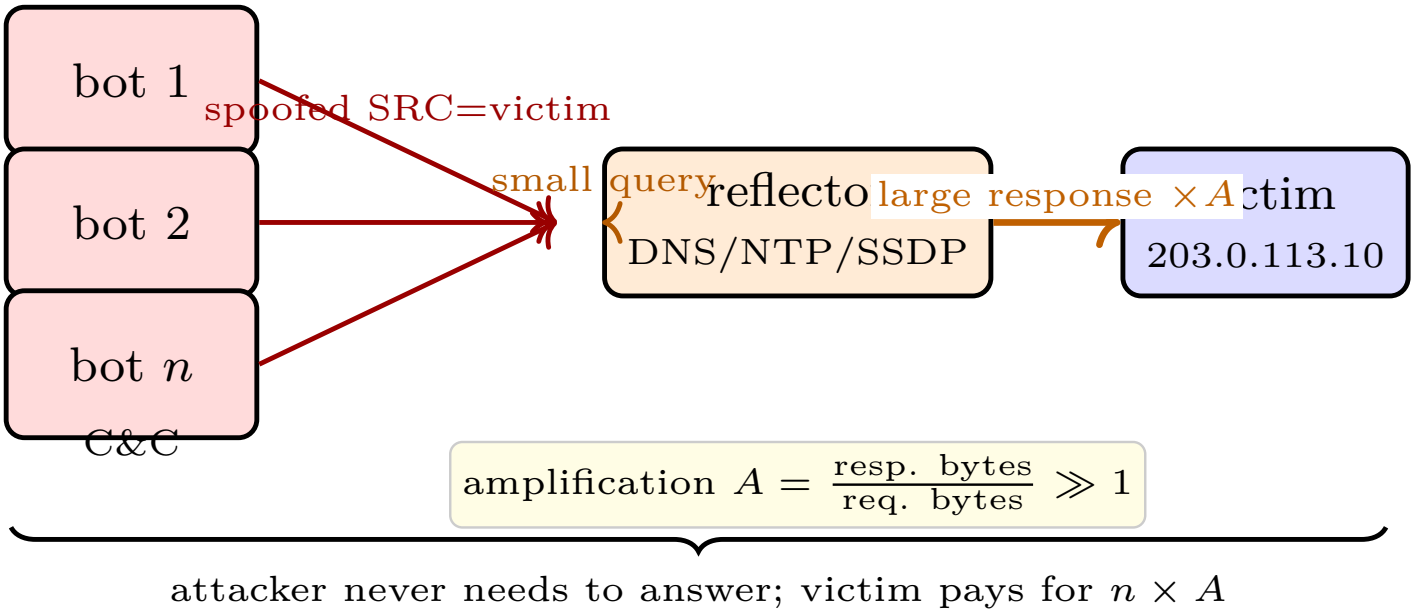


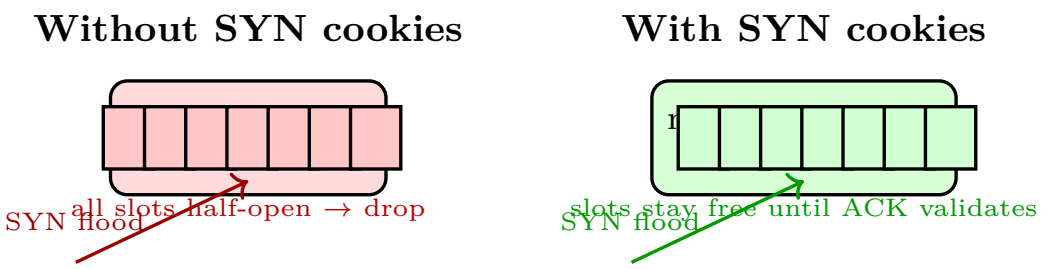
Figure 5.1: Amplification DDoS kill-chain. Bots send small spoofed queries; open reflectors reply with large responses to the victim. A of 20–500× is common with DNS/NTP.

Definition 5.1 (Amplification factor)**.** For a reflector R, A_R = (bytes in response)/(bytes in request) as seen by R. The victim sees $n \cdot A_R$ times the attacker's upstream. Open DNS with ANY or DNSSEC validates $A \approx 30$–60; NTP monlist reached $> 500\times$ before being disabled. Mitigation: close open reflectors and enforce ingress BCP38 anti-spoofing.

Botnets provide n; reflectors provide A. Mirai (2016) showed how IoT defaults create $n \approx 600$k; a single open DNS population still numbers in the millions. BCP38 (ingress filtering) would kill spoofed sources, but deployment is incomplete, so the defender must assume n and A are both large.

5.2 Protocol Exhaustion: The SYN Flood

TCP's three-way handshake keeps state after the first SYN. A server with backlog B (often 128–4096) allocates a transmission control block per half-open. A SYN flood sends SYNs from spoofed or bot IPs and never completes the handshake; the backlog fills, and legitimate SYNs are dropped with no answer.



cookie = HMAC(secret, IP/port/timestamp) in ISN: stateless until the ACK proves return routability

Figure 5.2: SYN backlog exhaustion vs. SYN-cookie rescue. Cookies trade a cryptographic check for per-connection state and keep the queue available for legitimate clients.

SYN cookies remove the trade: instead of storing state, the server encodes it into the initial sequence number $ISN = H(\text{secret}, \text{src/dst IP/port}, t) \| t \| MSS$. Only a final ACK that echoes a valid cookie recreates the connection. Cost: TCP options (window scaling, SACK) are limited during cookie mode, so Linux enables cookies only under pressure (`net.ipv4.tcp_syncookies=1`).

Rate and SYN policing complement cookies: `nft limit` or XDP/eBPF drops excessive SYNs per source or per prefix before they reach the stack, but aggressive policing also drops legitimate bursts (e.g., a campus NAT behind one IP). The right balance is cookies as safety net plus a loose per-/24 policer that triggers only during flood.

Listing 5.1: Lab-safe SYN flood with Scapy in an isolated netns (never run on production).

```
from scapy.all import IP, TCP, send
dst = "10.0.5.10"   # lab victim in the same netns
for i in range(200):
    pkt = IP(dst=dst)/TCP(dport=80, flags="S", seq=1000+i, sport=40000+i)
    send(pkt, verbose=0)
# Observe on victim: ss -t -a | grep SYN-RECV ; dmesg | grep "possible SYN flood"
# Mitigate: echo 1 > /proc/sys/net/ipv4/tcp_syncookies ; sysctl net.ipv4.
    tcp_max_syn_backlog
```

Listing 5.2: SYN policing before the stack: nftables and XDP intuition.

```
# nftables: rate-limit new SYNs, burst 40, then drop
nft add rule inet filter input tcp flags syn / syn,rst limit rate over 100/second burst 40
     packets drop
# Check cookie mode is on and backlog counters
sysctl net.ipv4.tcp_syncookies
cat /proc/net/netstat | tr ' ' '\n' | grep -i syncookies
# XDP/eBPF sketch: drop SYNs exceeding per-/24 counter with a BPF map (see Ch.3 eBPF stub)
```

5.3 Application Floods and Pulsing

An attacker who cannot fill the pipe can still burn the server. A single HTTP GET may trigger a DB query, a template render, and a TLS handshake. At scale, legitimate-looking requests become a cost-asymmetry weapon: the attacker sends cheap requests, the server does expensive work. TLS renegotiation, regex backtracking (ReDoS), and large-range HTTP Range headers are classic amplifiers at layer 7.

Pulsing exploits time instead of size. A 10 s burst at 10 Gbps followed by 50 s idle has an average that stays under a 5 Gbps threshold, yet each burst fills shallow buffers and triggers retransmissions. Detection needs short windows (1–5 s) plus a counter that remembers recent bursts. Carpet-bombing does the same in space: spreading traffic over many IPs in a /24 forces a prefix-wide filter that risks collateral damage.

Mitigation at L7 combines WAF signatures (block known expensive patterns), proof-of-work or CAPTCHA for suspicious clients, and autoscaling with a budget cap so scaling does not become a billing DoS. The cheapest fix is often to make the expensive endpoint cheaper: cache, paginate, and limit range headers.

Remark 5.1 (Why BCP38 is not enough). Ingress filtering (BCP38) would stop spoofed sources at the edge, but deployment is uneven: many access networks still forward packets with any source IP. ROV and SAV drafts push providers, yet an attacker needs only a few non-compliant ASes to launch an amplification flood. Defence must assume spoofing remains possible and combine edge filtering with reflector hygiene: disable open resolvers, turn off NTP monlist, and rate-limit SSDP/CHARGEN at the reflector itself.

Vector	**Amplification** A	**Bandwidth at reflector**	**Hygiene fix**
DNS ANY/DNSSEC	30–60×	open resolver, large TXT	RRL, close recursion
NTP monlist	> 500×	monlist enabled	upgrade, disable monlist
SSDP	20–30×	UPnP on WAN	disable WAN UPnP
Memcached	10 000–50 000×	unauth UDP 11211	firewall, auth

Table 5.1: Common reflectors ordered by amplification and the fix that removes them. Closing the reflector is cheaper than scrubbing its traffic forever.

A defender's cost model is bytes × time × collateral. Dropping at the victim is free but the link is already full; scrubbing costs per clean Mbps but preserves the link; Anycast needs standing PoPs but amortises over all customers. Budget for clean capacity, not just filter rules.

5.4 Scrubbing, Anycast, and the Playbook

When the access link is saturated, filtering at the victim is already too late. Three escalations move the fight upstream.

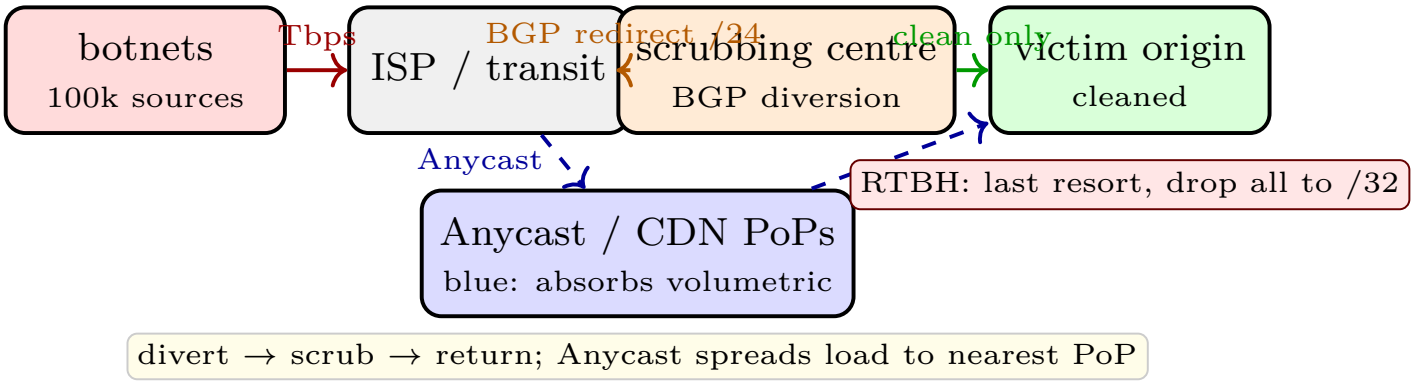


Figure 5.3: Upstream defences. Volumetric traffic is diverted via BGP to a scrubbing centre or absorbed by Anycast/CDN; clean traffic is re-injected via GRE/MPLS. Remote-triggered blackholing is the emergency brake.

Scrubbing centre: BGP announces the victim /24 from the scrubber with a more specific or community-tagged route, diverts traffic via GRE, filters (SYN validation, amplification signature, flowspec), and returns clean

traffic. *Anycast/CDN:* the same IP is announced from many PoPs; each attacker is routed to the nearest PoP and the total is partitioned. *Flowspec/RTBH:* BGP flowspec pushes 5-tuple filters to peers; remote-triggered blackholing drops all traffic to a /32 to save the rest of the network at the cost of the target.

Scrubbing adds latency (detour) and cost (clean bandwidth billed) but preserves availability; Anycast scales horizontally but only for stateless volumetric; RTBH is free and immediate but sacrifices the victim to protect neighbours. Choose by blast radius: RTBH for a single dispensable IP, scrubbing for a critical /24, Anycast as standing absorption for public web.

Playbook (detect→classify→filter→scale→post-mortem): *Detect* via NetFlow/sFlow spikes and SYN-RECV counters; *classify* as volumetric/protocol/app by ratio of bytes/PPS/flows; *filter* nearest the source (BCP38 + flowspec) and locally (cookies, WAF); *scale* by diverting to scrubber or Anycast; *post-mortem* with a timeline, IoCs, and costed fixes. Every step has a pre-written runbook and a named owner. Test it with a tabletop before the real flood; the first time you run a playbook should not be during the incident. Test the playbook without an attacker: inject a synthetic NetFlow spike, trigger the runbook, and time each handover. A playbook that has never been table-topped will fail its first real execution on a missing credential or a stale BGP community. Loop until clean, then post-mortem with timeline and owner.

5.5 Exercises

E5.1 **Taxonomy.** Give one concrete attack for each of volumetric, protocol, and application DDoS. For each, state the bottleneck it exhausts and why filtering near the victim may already be too late.

E5.2 **Amplification arithmetic.** A DNS ANY response is 3 000 B for a 60 B query ($A = 50$). 500 bots each send 2 Mbps of queries via reflectors. What rate hits the victim? Propose two fixes that cut A at the reflector rather than at the victim.

E5.3 **SYN flood lab.** In an isolated netns, use Listing 5.1 against a lab VM with `tcp_syncookies=0` and `=1`. Measure established vs. dropped legitimate connections with `ss -s` and `dmesg`. Explain the cookie's trade-off on TCP options.

E5.4 **Scrubbing decision.** A /24 hosts one critical API (needs 99.99%) and one blog. Volumetric flood targets the blog's IP. Compare BGP RTBH (/32) vs. scrubbing diversion vs. Anycast absorption on blast radius, time to divert, and cost. Which do you choose?

E5.5 **Playbook tabletop.** A pulsing SYN flood (10 s on, 50 s off) evades a 60 s-average rate limit. Write a detection rule using a 5 s window plus SYN-cookie activation, and draft the post-mortem section that prevents recurrence.

Remark 5.2 (Operational note). This design assumes continuous tuning, not a one-time deploy. Thresholds drift with traffic growth; retrain baselines monthly and after any architecture change. A threshold that was correct at 1 Gbps is noise at 10 Gbps. Log retention is a cost decision: 30 days of full NetFlow at 10 Gbps sampled 1:100 is ≈2 TB; justify it against the forensics window your IR team needs. Keep honeynet logs longer because they are tiny and high value. Every automation (auto-block, auto-quarantine) needs a manual override and an audit trail. An auto-block that triggers on a CDN PoP will outage your own users; require a second signal (e.g., honeynet + NetFlow) before blocking a /24.

Budget for clean capacity separately from filter rules: a 10 Gbps scrubbing contract that is never tested is a hope, not a plan. Negotiate burst pricing and test diversion quarterly, including the return path via GRE and the BGP community that triggers it. Measure collateral: a per-/24 policer that blocks a university NAT will drop hundreds of legitimate users for one attacker behind the same prefix. Prefer per-/32 plus reputation over per-prefix where flows allow, and log every block with its 5-tuple for appeal. Application teams own L7 cost: add a cache header, paginate expensive queries, and enforce a max Range length. The cheapest DDoS mitigation is often a code change that makes one request ten times cheaper, documented in the post-mortem as a permanent fix. Keep a public status page and a private war room. Volumetric floods are visible externally; a timely "we are diverting via scrubbing, expect +20 ms" reduces support tickets and pressures the provider to meet its SLA. Record every incident as a timeline: first NetFlow anomaly, first SYN-RECV spike, first WAF hit, diversion start, clean return, and customer impact. Cost the incident in clean bandwidth and engineering hours; that number funds the next mitigation. Do not forget IPv6: many scrubbers and flowspec deployments still lag on v6. An attacker who cannot saturate v4 may try v6; ensure RPKI, BCP38, and SYN cookies are enabled on both stacks and that Anycast covers v6 as well. The goal is graceful degradation, not perfect absorption. If the scrubber is overwhelmed, shed the lowest-value traffic first (blog, images) and protect the API with priority queuing. Document the shed order in advance so the on-call does not decide under fire. Finally, close reflectors at the source: scan your own address space for open DNS, NTP, SSDP, and memcached and close them. You are both potential victim and unwitting amplifier; hygiene is a community defence. Automate reflector hygiene with weekly scans: run `zmap` for UDP 53/123/1900 from a measurement net and notify owners of open reflectors. Community scanning reduces the amplifier population faster than any scrubber. Separate infrastructure cost from attack cost: an attacker pays for bots and spoofed queries, the defender pays for clean bandwidth and on-call hours. Budget the incident in both, and show the cost of not closing reflectors. Test scrubbing diversion with a benign prefix: announce a test /24 via the scrubber, measure latency via `mtr`, and verify return via GRE. Do this monthly so the BGP community and tunnel are not stale when needed. Document the RTBH one-pager: which peer, which community, which /32, and who can authorize it. Under stress, an unpracticed RTBH will blackhole the wrong prefix and extend the outage. Prefer BCP38 plus SAV at the edge: source-address validation is the only cure for spoofed amplification. Support MANRS and require upstream peers to filter, even if compliance is incremental. For SYN floods, monitor retransmits: a legitimate client that retransmits SYN but never reaches ESTABLISHED is evidence of backlog fill, not client failure. Graph retransmits alongside SYN-RECV. Application teams should own L7 budgets: each endpoint gets a cost tag (DB queries, external calls) and a latency SLO. An endpoint whose p95 exceeds 500 ms under normal load is a DoS amplifier waiting to happen. Do not autoscale without a ceiling: a 10 Tbps flood that triggers autoscaling will become a 10 Tbps bill. Cap scale and shed low-value traffic when cost exceeds a threshold; document the shed order. Honeynet hits during a DDoS are not noise: an attacker who floods one prefix may probe another for a backdoor. Correlate volumetric spikes with honeynet probes in the same time window. Post-mortem must include reflector data: which reflectors were abused, their ASes, and whether abuse reports were sent. Closing one reflector is more valuable than scrubbing its traffic ten times. IPv6 is not exempt: many DDoS tools now support v6. Ensure the scrubber, flowspec, and Anycast cover your v6 prefix and that SYN cookies are enabled for v6 as well. Keep a public runbook for customers: "under DDoS, we will divert via scrubbing, expect +15 ms, status at status.example.com". Transparency reduces support load during the flood. Energy and compliance matter: a scrubbing centre that diverts all traffic through another continent may breach data residency. Choose scrubber geography with legal, not just network, input. Rehearse pulsing detection with synthetic bursts: inject 5 s, 20 Gbps bursts every 50 s and verify the short-window detector fires. Long-average

detectors will miss it; short-window plus burst counter will not. Test scrubbing at L7 as well: a WAF rule that blocks the expensive "search?q=*" pattern during flood can cut CPU while the scrubber cuts bandwidth. Both are needed for app-layer floods. Keep a darknet sensor in the same prefix as the critical API: any scan that hits the darknet before the API is a pre-attack signal, and its IoCs can be fed to flowspec before the flood peaks. Coordinate with upstream: a single email to the transit with flowspec and RTBH communities is faster than any local filter when the link is already full. Pre-register the communities. Budget the post-mortem: every incident should produce one permanent fix (close a reflector, add a cache, tune a WAF) and one detection improvement (new SIEM rule or shorter window). Without a fix, the next flood is the same flood. Document the playbook owner and backup owner: an incident that starts when the primary is on a flight will fail if the backup does not know the BGP community or the scrubber portal login.

Chapter Notes

DDoS taxonomy and reflector amplification follow Rossow (2014) and the classic DNS/NTP/SSDP studies; BCP38 ingress filtering is Ferguson–Senie (RFC 2827). SYN cookies are Bernstein–Schenk (1996) and Linux `tcp_syncookies`; XDP/eBPF filtering is in the Cilium/eBPF docs. Scrubbing, flowspec (RFC 5575), RTBH (RFC 3882), and Anycast/CDN absorption are covered in NIST SP 800-189 and Arbor/WIT reports. The playbook mirrors SANS DDoS and Google SRE incident lifecycle.

Chapter 6

Wireless Security: WEP, WPA2, WPA3 and Evil Twins

> *"If the medium is air, the adversary owns the air around you."* — A cable is a private hallway; Wi-Fi is a shout across a café. Anyone nearby can listen, and anyone who shouts louder can pretend to be the café.

From zero: no wireless background assumed. We start from a radio broadcast where every nearby receiver hears every frame. From that intuition we ask: how do we give a private conversation over a public shout, why the first attempt (WEP) failed catastrophically, how WPA2 fixed it but still left a handshake crack, and how WPA3 tries to fix that too. One picture at a time: cipher, handshake, impersonation.

Learning Outcomes

After this chapter you will be able to:

(L1) explain why wireless amplifies eavesdropping, injection and impersonation compared to wired;

(L2) describe WEP's RC4/CRC design and enumerate its three fatal flaws;

(L3) trace the WPA2 4-way handshake, CCMP's AES-CCM, and the KRACK reinstall;

(L4) compare WPA3-Personal (SAE), WPA3-Enterprise 192-bit, OWE and PMF with WPA2;

(L5) design and detect an evil-twin / rogue AP attack and propose mitigation.

6.1 Air Is Broadcast

On Ethernet a frame traverses a wire between two ports; on 802.11 it radiates. Any antenna in range ($\approx 30\,\text{m}$ indoors, $> 200\,\text{m}$ with a dish) receives every frame, regardless of destination. The header (BSSID, source/destination MAC, sequence number) is never encrypted, even when the payload is. An attacker can

passively map SSIDs, count clients, measure signal to locate devices, and inject frames by waiting for a clear channel. Confidentiality, integrity and authenticity must be retrofitted onto this broadcast.

A *master secret* (pre-shared key or enterprise credential) must be stretched into fresh session keys per association, and those keys must be proven without revealing them over the air. That proof is the handshake.

6.2 WEP: A Case Study in How Not To

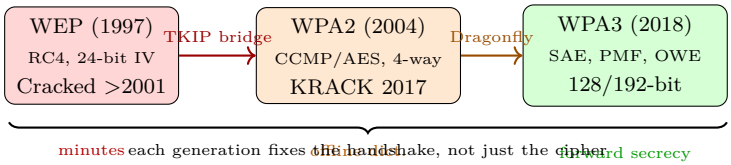
WEP (1997) used RC4 with a 40- or 104-bit key plus a 24-bit IV prepended in clear: keystream = RC4(key $Vert$ IV), ciphertext = plaintext $\oplus$ keystream, integrity = CRC-32 (linear, not cryptographic). Three breaks, each fatal alone:

Small IV, no replay protection 24 bit $\rightarrow 2^{24} \approx 16$ M values. A busy AP cycles the IV in hours; collisions reuse the same keystream. XOR of two ciphertexts with the same IV is XOR of plaintexts—English text breaks quickly.

RC4 weak keys + related-key The first bytes of RC4 keystream leak key bits (Fluhrer–Mantin–Shamir, 2001). With $\approx 40\,000$ frames an attacker recovers the key passively.

Linear CRC Flipping a bit and fixing the CRC is trivial; integrity checks nothing against an active attacker.

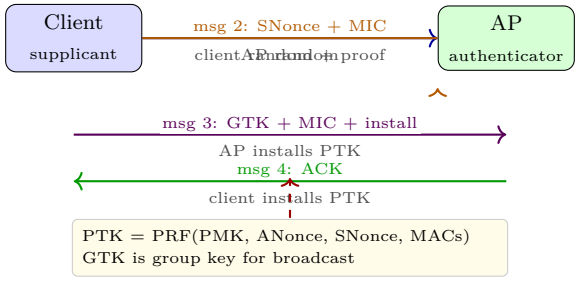
Tools like `aircraft-ng` automated this to minutes by 2005. WEP is not "weak"; it is broken by design. Never use it, even for "low-value" traffic.



WPA (2003) was a TKIP band-aid that still used RC4; WPA2 (2004) finally mandated AES.

6.3 WPA2: Handshake and CCMP, Then KRACK

WPA2-Personal derives a Pairwise Master Key PMK = PBKDF2(passphrase, SSID, 4096). The 4-way handshake proves possession of the PMK and negotiates fresh PTK/GTK without sending the PMK over the air, and installs the key for CCMP (AES in CCM mode: authenticated encryption).



CCMP then encrypts each 802.11 data frame with a 48-bit packet number (PN) as nonce: replay is detected when PN does not increase, and tampering is caught by the CCM MIC. This fixed WEP's crypto.

KRACK (Vanhoef–Piessens, 2017) did not break AES; it broke the *handshake state machine*. By replaying msg 3, the attacker forced the client to reinstall an already-used PTK, resetting PN and keystream. With keystream reuse, the attacker could decrypt and inject. The fix is to forbid reinstall of the same key and to use a single PN counter across installs. PMF (802.11w) helps: it protects deauthentication frames that KRACK abused to force reconnects.

Offline dictionary remains WPA2-Personal's practical weakness. The 4-way handshake's MIC gives an offline verifier for passphrases; a captured handshake plus a wordlist (or GPU brute-force at $\approx 10^9$ PBKDF2/sec on modern GPUs) cracks weak passwords. Enterprise mode avoids this by using 802.1X/EAP-TLS with per-user credentials.

Listing 6.1: Capturing a WPA2 handshake and testing a passphrase list.

```
# Put card in monitor, capture handshake on channel 6
sudo airmon-ng start wlan0           # -> wlan0mon
sudo airodump-ng -c 6 --bssid AA:BB:CC:DD:EE:FF -w capture wlan0mon
# Deauth a client to force a fresh handshake (lab only, with permission)
sudo aireplay-ng --deauth 5 -a AA:BB:CC:DD:EE:FF wlan0mon
# Crack offline: only works for weak passphrases
aircrack-ng -w rockyou.txt capture-01.cap
# Mitigation check: is PMF required? (802.11w)
iw dev wlan0 scan | grep -i "PMF"
```

6.4 WPA3: SAE, Forward Secrecy and OWE

WPA3-Personal replaces the 4-way handshake's PSK proof with *Simultaneous Authentication of Equals* (SAE, Dragonfly): a zero-knowledge password-authenticated key exchange. An attacker who captures the handshake learns nothing for offline dictionary; they must *interact* for each guess, and the AP can rate-limit. SAE also gives *forward secrecy*: compromising the password later does not decrypt past captures. Transition mode (WPA2+WPA3 on the same SSID) keeps compatibility but downgrade attacks are possible if not configured to prefer WPA3.

For open networks, WPA3 adds *Opportunistic Wireless Encryption* (OWE, RFC 8110): unauthenticated encryption—no passphrase, but every client gets a fresh key so passive sniffing is defeated. The café AP finally encrypts even when there is no password; authentication is still absent (evil twin still possible).

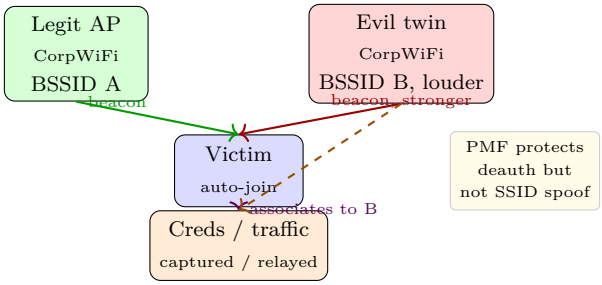
WPA3-Enterprise offers a 192-bit suite (GCMP-256, HMAC-SHA-384) and mandates PMF. Together, SAE + PMF + OWE close the three WPA2 gaps: offline dictionary, deauth, and open-network sniffing.

Remark 6.1 (Dragonblood and transition pains). WPA3 did not ship flawlessly. Dragonblood (Vanhoef–Ronen, 2019) showed side-channel and downgrade issues in early SAE implementations: timing and cache attacks leaked password bits, and a transition-mode AP that offered both WPA2 and WPA3 could be downgraded to the weaker handshake by spoofing beacons. The fixes were firmware upgrades and an anti-clogging token plus constant-time hunting-and-pecking. Lesson: even a formally verified handshake (SAE is based on a balanced PAKE) can fail in code that is not constant-time. Deploy WPA3 only with patched firmware and set `sae_require_mfp=1` to

forbid downgrade where possible. Transition mode is a bridge, not a destination.

6.5 Evil Twin and Rogue AP

An *evil twin* copies a legitimate SSID/BSSID (often with higher TX power) and lures clients that auto-join by SSID alone. With `hostapd` plus `dnsmasq` the attacker runs a full AP on a laptop, serves a captive portal, and relays or strips traffic. Enterprise (EAP-TLS) resists the twin because the client validates the *server's certificate*, not just the SSID. Pre-shared-key networks have no such anchor.



Detection: monitor for duplicate BSSIDs with divergent capabilities (PMF required vs. not, differing RSSI jumps, unexpected channel, new OUI). Enterprise tools (WIDS/WIPS) triangulate rogues; on Linux:

Listing 6.2: Rogue AP detection with signal and capability anomaly.

```
# Scan and flag SSIDs seen with >1 BSSID or capability mismatch
sudo iw dev wlan0 scan | awk '/BSS/{b=$2} /SSID:/{s=$2} /RSN:/{r=$0} {print b,s,r}' | sort
    | uniq -c | awk '$1>1'
# Continuous monitor: alert on deauth floods (PMF should protect)
sudo tshark -i wlan0mon -Y "wlan.fc.type_subtype==12" -T fields -e wlan.sa -e wlan.da |
   sort | uniq -c | sort -nr | head
# Prefer WPA3/OWE and pin expected BSSID where possible
# wpa_supplicant: bssid_allowlist=aa:bb:cc:dd:ee:ff  (lab pinning)
```

Mitigations ladder: for new deployments use WPA3-Enterprise with EAP-TLS (certificate pinning kills the twin), or WPA3-Personal with a strong random 20+ character passphrase and PMF required. For existing WPA2-PSK, rotate to a long passphrase, disable WPS, enable PMF if clients support it, and educate against auto-joining open or known SSIDs. For open hospitality, deploy OWE so even unauthenticated encryption raises the bar.

6.6 Putting It Together: Hardening Checklist

Wireless hardens in layers, mirroring the firewall ladder of Ch. 2. Use this checklist when you audit a deployment:

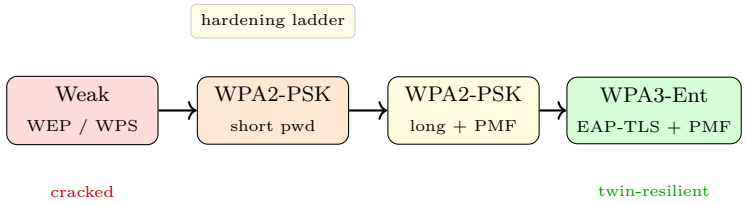
1. **Choose the suite per SSID.** Employee: WPA3-Enterprise/EAP-TLS, 192-bit, PMF required. Guest: OWE (or WPA3-Personal with a daily rotating 20-char passphrase displayed at reception). Never mix WEP/WPA with the same SSID.

2. **Require PMF and disable downgrade.** Set `pmf=2` (required) on the AP; clients that cannot do PMF are upgraded, not accommodated. In transition mode, prefer WPA3 and log WPA2 associations for upgrade tracking.

3. **Manage the secret.** For PSK, generate with `pwgen 24 1` and rotate quarterly; for Enterprise, provision per-user certificates via MDM and revoke on off-boarding. Disable WPS (PIN is brute-forceable in $<$10 hours).

4. **Detect rogues continuously.** Deploy a WIDS sensor per floor (or use AP's built-in WIDS) that alerts on new BSSID+SSID pairs, PMF mismatch, and deauth floods. Feed alerts to the SIEM (Ch. 7).

5. **Segment wireless from wired.** Terminate Wi-Fi on a controller in the DMZ; firewall the controller→core at L3. A compromised wireless client should not see the DB VLAN without an additional VPN/802.1X hop.



Listing 6.3: Hardening wpa_supplicant and hostapd for WPA3 + PMF.

```
# wpa_supplicant.conf -- prefer WPA3, require PMF, pin BSSID
network={
    ssid="CorpWiFi"
    key_mgmt=SAE                # WPA3-Personal; use WPA-EAP for Enterprise
    sae_password="correct-horse-battery-staple-24!"
    ieee80211w=2                # 2 = require PMF, 1 = optional
    bssid_allowlist=aa:bb:cc:dd:ee:ff
    scan_freq=2412 2437 2462
}
# hostapd.conf -- WPA3-Personal, PMF required, disable WPS
interface=wlan0
ssid=CorpWiFi
wpa=2
wpa_key_mgmt=SAE
wpa_passphrase=correct-horse-battery-staple-24!
ieee80211w=2
sae_require_mfp=1
wps_state=0                     # disable WPS PIN
```

Remark 6.2 (Usability vs. security). Every extra click pushes users toward workarounds (hotspots, tethering). OWE is the pragmatic win for open venues: zero friction, yet passive sniffing is defeated. For employee Wi-Fi, the MDM that silently provisions the EAP-TLS certificate is worth more than a 30-character PSK that users write on sticky notes.

Remark 6.3 (Wi-Fi 6E and the 6 GHz mandate). At 6 GHz (Wi-Fi 6E) the Wi-Fi Alliance mandates WPA3; WPA2 is not allowed. Greenfield 6 GHz deployments are thus WPA3-only by regulation, sidestepping transition-mode downgrade entirely. If you control the band, deploy there first and keep 2.4/5 GHz as a legacy bridge with PMF logging. The 6 GHz band is also where OWE becomes the default for "open" hospitality: encrypted by default, even when authentication is absent.

Measurement tip: Run `iw dev wlan0 station dump` on associated clients to see negotiated cipher (CCMP vs. GCMP) and PMF status; `hostapd_cli sta` shows the AP's view. A mismatched `CCMP` vs. `GCMP-256` on a WPA3-Enterprise SSID is a client that needs a driver update before you can enforce 192-bit.

6.7 Exercises

Exercise 6.1. WEP uses a 24-bit IV. A busy 802.11g AP sends 700 frames/sec. How long until IV collisions are expected (birthday bound $\approx \sqrt{2^{24}}$)? Why does this break confidentiality even though RC4 itself may be keyed correctly?

Exercise 6.2. Trace the WPA2 4-way handshake labelling ANonce, SNonce, PMK, PTK and GTK. At which message does the AP install the PTK? Explain how replaying message 3 in KRACK resets the packet number and why that leads to keystream reuse in CCMP.

Exercise 6.3. An 8-character lowercase password is used with WPA2-Personal (PBKDF2 4096 iterations). Estimate the search space and discuss whether a captured 4-way handshake can be cracked offline on a GPU testing 10^6 passphrases/sec. How does WPA3 SAE change the answer? Include forward secrecy.

Exercise 6.4. Design an evil-twin lab (with permission): (a) write a `hostapd.conf` that clones an existing WPA2 SSID, (b) explain why a client that auto-joins open SSIDs is easier to bait than one that requires WPA3-Enterprise certificate validation, (c) list three detector signals a WIDS would use to flag your twin.

Exercise 6.5. A hotel offers `HotelWiFi` as open + captive portal. Compare (i) open, (ii) OWE, and (iii) WPA3-Personal with a per-room password for confidentiality, evil-twin resilience and usability. Recommend a deployment with PMF and explain the residual risk of a twin that also offers OWE.

Lab note: In a permissioned testbed, compare `aircrack-ng` time on WPA2-PSK (`rockyou.txt`, 14M) vs. WPA3-SAE where the same capture yields no offline verifier; the tool will report "no valid handshake" and each guess must be online, throttled to ≈ 10/s by the AP. Transition captures also reveal the cost of crypto: CCMP adds 16 B MIC + 8 B header per MPDU; at 1500 B MTU the overhead is $\approx 1.6\%$ but the per-packet AES operation at $\sim$1 Gbps is negligible on modern SoCs with AES-NI.

Chapter Notes

WEP breaks are Fluhrer–Mantin–Shamir (2001) and Tews–Weinmann–Pyshkin (2007); CCMP and the 4-way handshake are 802.11-2016. KRACK is Vanhoef–Piessens (2017, CCS) and its PMF discussion is 802.11w-2009. WPA3/SAE/OWE are Wi-Fi Alliance (2018) and RFC 8110; 192-bit suite is Suite-B. Evil-twin tooling is `hostapd`, `airbase-ng` and `wifiphisher`; detection follows 802.11 – Physical layer DoS and rogue-AP surveys (Bahl et al.).

Chapter 7

Monitoring, Forensics and Zero Trust

"You cannot hunt what you never collected." — Detection starts before the incident, in the quiet decision to keep the right logs and to distrust the network you already own.

From zero: seeing, trapping, and trusting nothing. We start from a mirrored packet, compress it into a flow, store it for a hunt, lure the attacker into a fake service, and finally stop trusting the wire at all. Intuition (pipe → tap) → collection → deception → zero trust. No prior SOC background assumed.

Learning Outcomes

After this chapter you will be able to:

(L1) capture and dissect traffic with `tcpdump`/Wireshark and summarise it as NetFlow/IPFIX;

(L2) deploy honeypots/honeynets and DNS/BGP hardening (DNSSEC, RPKI) as network sensors;

(L3) run a forensics workflow: preserve → timeline → IoC hunt → attribute;

(L4) query a SIEM (Splunk SPL) and build visibility baselines for alerting;

(L5) contrast perimeter (castle-moat) with zero trust (NIST 800-207) and map to ZTNA.

7.1 Seeing the Wire: Capture to SIEM

A tap copies light (passive, no latency); a SPAN/mirror copies in the switch ASIC (cheap, may drop under load). Either feeds capture.

`tcpdump` (CLI, `libpcap`) and Wireshark (`pyshark`) dissect the copy; NetFlow/IPFIX summarise it. A flow is the 5-tuple plus counters: bytes, packets, start, duration, flags. Collectors (e.g., `nfdump`, `pmacct`) export flows to a SIEM, which correlates IDS alerts, flows, and host logs on one timeline. Sampling (1:100) is required at 10 Gbps.

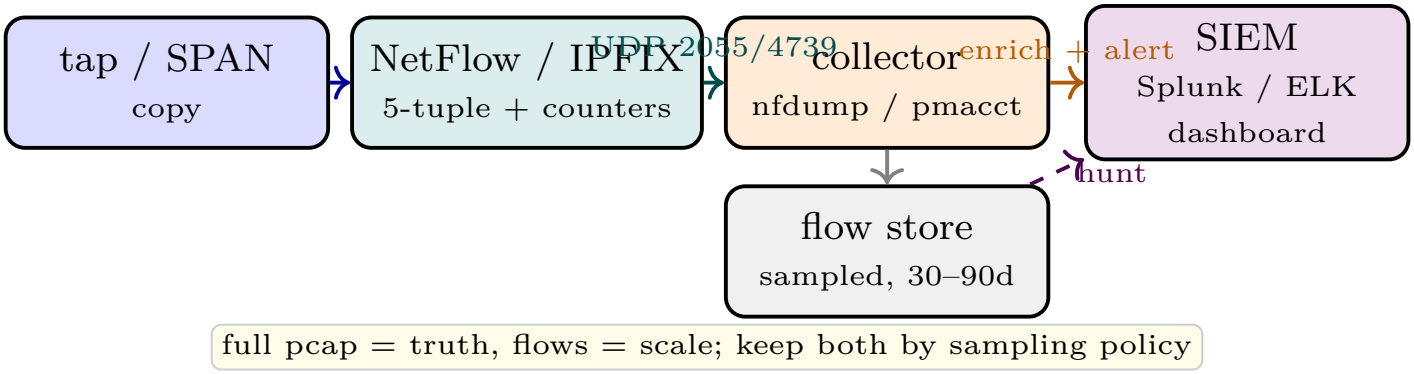


Figure 7.1: Visibility pipeline. A passive copy becomes flows at the collector and incidents in the SIEM. Sampling and retention policy determine what you can hunt tomorrow.

Baselining turns collection into detection: per-subnet bytes, per-service entropy, and per-host outbound flows per hour. A host that normally opens 30 flows/hour and suddenly opens 2 000 to port 443 on many destinations is beaconing until proven otherwise.

Listing 7.1: From packet to flow to hunt: tcpdump, nfdump, and Splunk SPL.

```
# Capture: 60s of DNS and SYN on the DMZ mirror
tcpdump -i eth1 -w dmz.pcap -G 60 -W 1 'port 53 or tcp[tcpflags] & tcp-syn != 0'
# NetFlow: synthesise flows from the pcap, then top talkers
nfdump -r dmz.nf -o extended -s ip/bytes -n 10   # top 10 by bytes
nfdump -r dmz.nf 'port 53' -o csv | head          # DNS flows for hunting
# Splunk SPL: beaconing and DNS tunnelling intuition
# | tstats count where index=netflow by src_ip, dst_ip
# | where count > 1000 | sort -count
# search index=dns query_length > 80 | stats count by src_ip | where count > 200
```

7.2 Deception and Hardened Naming/Routing

A honeypot is a fake vulnerability that is cheap for you and expensive for the attacker to verify. Low-interaction (e.g., `cowrie` SSH) emulates a service; high-interaction runs a real instrumented host. A honeynet is a subnet of honeypots with a darknet (unused address space) around them. Any packet there is suspicious by construction; an alert from the honeynet has far higher precision than one from production.

DNS and BGP are network infrastructure worth hardening as sensors. DNSSEC signs zones (RRSIG/DNSKEY) so resolvers can validate answers and detect cache poisoning; RPKI signs BGP route origins (ROA) so routers can reject hijacks. Neither encrypts, but both make spoofing detectable and therefore huntable.

Listing 7.2: Low-interaction honeypot and DNS/BGP validation checks.

```
# Cowrie SSH honeypot (low interaction, JSON logs to SIEM)
pip install cowrie && cowrie start --listen 0.0.0.0:2222
tail -f ~/cowrie/var/log/cowrie/cowrie.json | jq '.eventid, .src_ip, .input'
# DNSSEC validation check
dig +dnssec example.com | grep -i "ad flag"    # AD=1 means validated
delv @8.8.8.8 example.com                       # verbose DNSSEC chain
# RPKI origin validation (routinator / rpki-client)
```

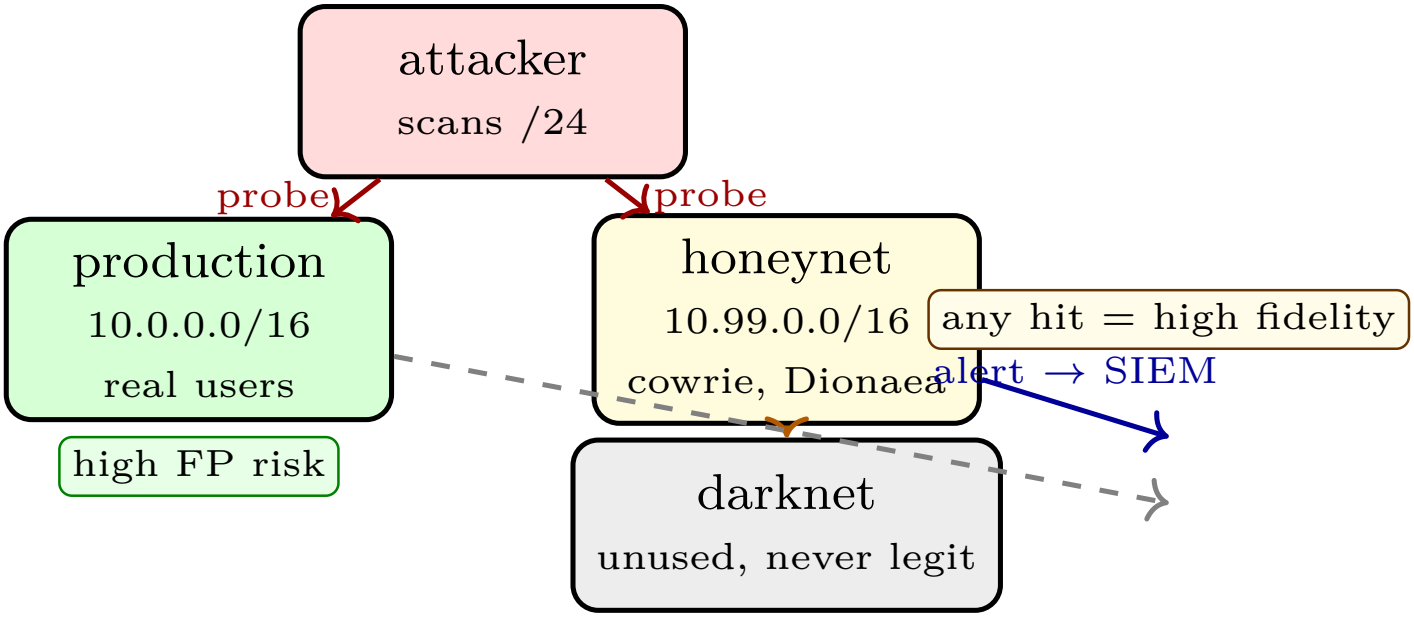


Figure 7.2: Honeypot vs. production. Production alerts are noisy; a darknet/honeynet has no legitimate traffic, so any contact is an attacker to investigate. Logs feed the same SIEM with higher priority.

```
rpki-client -v && bgpq3 -b -4 AS13335          # fetch ROAs, build prefix filter
```

Preserve forensics discipline: image with `dd` plus hash, freeze clocks (NTP), and keep a chain of custody. Then build a timeline (`plaso/log2timeline`, Zeek logs, auth logs), hunt IoCs (hashes, JA3, beacon intervals), and attribute with care: infrastructure is cheap to rotate, TTPs are harder to fake.

7.3 Forensics: Preserve, Timeline, Hunt

Forensics is not incident response; it is the evidence discipline that makes response defensible. *Preserve* means bit-for-bit imaging (`dd if=/dev/sda | sha256sum`), freezing clocks, and chain-of-custody notes: who collected, when, hash. Volatility or `LiME` captures RAM before it is lost. Write-blockers prevent the investigator from becoming the attacker.

Timeline merges sources on one clock: Zeek `conn.log` (every flow), `auth.log` (every login), EDR process trees, and firewall allows. `plaso` (`log2timeline`) normalises timestamps; even one skewed host (NTP off by minutes) creates false causality. Hunt next: pivot on IoCs (hash, IP, JA3, domain) across the timeline, not just in one log. A single beacon at 3 a.m. to a new AS is interesting; three such beacons spaced exactly 300 s apart is attribution-grade.

Remark 7.1 (Attribution caution). Infrastructure is cheap to rotate; TTPs are not. An IP that was the C&C last week may be a compromised home router today. Correlate TTPs (beacon interval, User-Agent, DNS pattern) plus human context before naming an actor. False attribution is a second incident.

A minimal hunt query: start from a honeynet hit, extract its source IP and JA3, search NetFlow for prior flows with the same JA3, then check which production host contacted that IP before the honeynet did. That host is patient zero until proven otherwise.

7.4 Zero Trust: From Castle-Moat to Mesh

A perimeter assumes inside is safe; compromise of one host then grants lateral movement. Zero trust (NIST SP 800-207) assumes the network is hostile and checks every flow as if it crossed the Internet: authenticate the user and device, authorise per resource, encrypt per flow, and log centrally. The enterprise becomes a set of micro-perimeters around each service, with a policy engine that grants short-lived access.

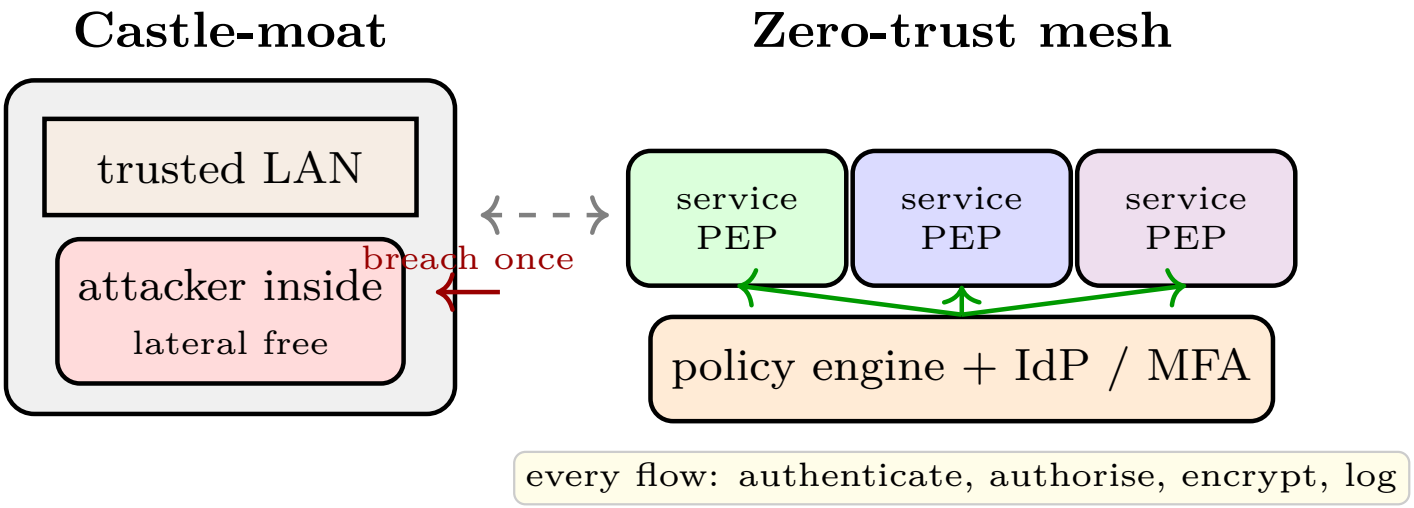


Figure 7.3: Perimeter vs. zero trust. The castle trusts by location; the mesh trusts by identity and checks per flow at a Policy Enforcement Point. Breach blast radius shrinks from the whole LAN to one resource.

NIST 800-207 components: Policy Engine (decides), Policy Administrator (provisions), Policy Enforcement Point (PEP, enforces on the data path), plus IdP, device inventory, and continuous diagnostics. ZTNA replaces VPN's coarse "inside" with per-app tunnels (often WireGuard or mTLS) and removes standing privilege: access expires in minutes, not months. Migration is incremental: identify crown jewels, put a PEP in front, require MFA and device posture, then expand the mesh.

Migration is four incremental cuts, not a flag day. (1) Inventory: list every service, data store, and current trust boundary. (2) Crown jewels first: put a PEP in front of the DB and require MFA plus device posture. (3) Expand the mesh service by service; each cut adds one PEP and one policy, old VPN routes are pruned. (4) Continuous verification: device health, user risk, and flow baselines feed the policy engine; a compromised device loses access in minutes, not at the next password rotation.

ZTNA tunnels are often WireGuard or mTLS per app, not one flat network. A user who can reach the wiki should not automatically reach the build system. Micro-segmentation is the network realisation of least privilege from Ch. 2, now enforced per flow rather than per VLAN.

7.5 Exercises

E7.1 **Baseline.** From one hour of NetFlow for `10.0.0.0/16`, compute per-host flows/hour and bytes/flow. Flag a host that jumps from 30 to 2 000 flows/hour to many /24s. What alert threshold would you set to keep false positives $< 5\%$?

E7.2 **Honeypot triage.** A cowrie log shows `root/admin` logins from `203.0.113.77` trying `wget`

`http://203.0.113.77/mips`. Is this automated? List three IoCs to hunt in SIEM and one action to block the follow-on.

E7.3 **DNSSEC/RPKI.** A resolver without DNSSEC sees a poisoned A record; one with DNSSEC sees it. Explain the validation that fails (RRSIG) and similarly how an RPKI-invalid BGP announcement is distinguished from a legitimate one.

E7.4 **Forensics timeline.** A host was compromised at T_0. You have Zeek `conn.log`, auth.log, and cowrie hits. Sketch preserve→timeline→IoC→attribute for the first 30 minutes and name the artefact that proves lateral movement vs. inbound scan.

E7.5 **Zero trust map.** Map a VPN-based remote access (castle-moat) to NIST 800-207: where are the PE, PA, and PEP, and what changes when you replace it with ZTNA per-app tunnels? Name one usability win and one deployment pitfall.

Remark 7.2 (Operational note). This design assumes continuous tuning, not a one-time deploy. Thresholds drift with traffic growth; retrain baselines monthly and after any architecture change. A threshold that was correct at 1 Gbps is noise at 10 Gbps. Log retention is a cost decision: 30 days of full NetFlow at 10 Gbps sampled 1:100 is ≈2 TB; justify it against the forensics window your IR team needs. Keep honeynet logs longer because they are tiny and high value. Every automation (auto-block, auto-quarantine) needs a manual override and an audit trail. An auto-block that triggers on a CDN PoP will outage your own users; require a second signal (e.g., honeynet + NetFlow) before blocking a /24.

Retention is policy, not storage: 90 days of sampled NetFlow at 1:100 on a 10 Gbps link is roughly 2 TB with compression, while full pcap for the same window is petabytes. Keep full pcap for the last 72 h on a ring buffer and flows for 90 days; justify the window against your mean time to detect. Normalize before you correlate: `log2timeline` plus Zeek and EDR timestamps must share NTP. A host whose clock drifts by 2 minutes will appear to beacon after the C&C was blocked, inverting causality. Alert on NTP skew as a sensor health check. Honeynet fidelity comes from isolation: the honeynet VLAN must have no route to production and no legitimate user should ever learn its address. Any hit is then high priority by construction. Mirror the same SIEM rules but with lower thresholds for the honeynet and page on first hit. DNSSEC and RPKI are validation, not encryption: DNSSEC proves the answer was not tampered, RPKI proves the BGP origin is authorized. Both fail closed only if you enforce validation; otherwise they are merely observable. Log validation failures and alert on RPKI-invalid or DNSSEC-bogus at the resolver. Baseline per VLAN, not globally: a DMZ web server that normally serves 10 k flows/hour looks different from a management jump host that serves 20. Per-VLAN baselines reduce false positives and let a 5 flows/hour beacon stand out on a quiet host. Splunk cost control: index only what you hunt. Drop debug logs, sample health checks, and keep honeypot JSON at full fidelity. A SIEM that is 90% noise will be ignored; a SIEM that is 10% honeynet and baseline breaches will be watched. Zero-trust rollout needs a fallback: when the policy engine is unreachable, fail closed for crown jewels and fail open with alerting for low-value wikis. Document the fallback per PEP and test it by killing the PA during a tabletop. Device posture is the unsung control: a PEP that checks MFA but not patch level will grant a compromised laptop the same access as a clean one. Integrate EDR health into the policy engine and revoke short-lived certs when health drops. Tune thresholds after every major release: a new video endpoint that streams 10 MB per request will shift the per-host bytes baseline and create false positives if the SIEM still uses the old mean. Tag flows with business context: “prod-web”, “guest-wifi”, “build”. A spike tagged “guest-wifi” at 2 a.m. is less urgent than the same spike on “prod-db”; context prevents alert fatigue. Keep the

darknet small but representative: a /24 darknet inside a /16 is enough to see scans without consuming address space. Route it to the honeynet collector and alert on any SYN to it. Test DNSSEC validation in staging: a misconfigured trust anchor will mark legitimate domains as bogus and break resolution. Monitor `SERVFAIL` rate after enabling validation and have a rollback key. RPKI ROA expiry is an outage: if your ROA expires, peers that validate will drop your prefix. Automate ROA renewal and alert 30 days before expiry; keep a non-validating peer for emergency reachability. Use JA3 and JA3S for TLS fingerprinting, but remember they are brittle: a client update changes the fingerprint. Combine JA3 with destination and timing, not alone, to avoid false attribution. Sampling must be consistent across collectors: if one router samples 1:100 and another 1:1000, top-talker comparison is meaningless. Standardise sampling and annotate it in every dashboard. Encrypt log transport: NetFlow and syslog over UDP are trivial to spoof. Use TLS-wrapped export (IPFIX over TLS) or at least HMAC the feed; otherwise an attacker can inject fake flows to distract the SOC. Practice evidence preservation under pressure: have a one-page checklist (image, hash, chain of custody) laminated at the jump host. During an incident, people skip steps unless the checklist is physical. Rotate honeypot credentials weekly: if the honeypot always uses `admin:password`, attackers learn to ignore it. Use realistic but fake credentials and monitor which ones are tried first. Correlate across time windows: a beacon every 300 s will be missed by a 60 s window but caught by a 10 minute autocorrelation. Use both short and long windows in the SIEM. Document data sources per alert: "this alert fired because NetFlow + Suricata SID 1000001 + honeynet hit" is actionable; "anomaly score high" is not. Provenance builds trust in the SIEM. Limit honeynet outbound: a high-interaction honeypot that is allowed to attack others becomes liability. Rate-limit and sinkhole outbound from the honeynet at the hypervisor. Baseline DNS query length and qtype distribution: DNS tunnelling uses long TXT or NULL queries. A host that normally queries A/AAAA and suddenly queries 80-char TXT is exfiltrating until proven otherwise. Keep a canary token in the honeynet: a fake AWS key or cookie that, when used, triggers an alert. Its use proves the attacker exfiltrated and tried to reuse, not just scanned. Integrate threat intel as context, not as blocklist: an IoC that is two weeks old is likely reassigned. Use intel to pivot hunts, not to auto-block, unless freshness is proven. Score alerts by precision: honeynet = 0.99, NetFlow baseline breach = 0.30, IDS signature = 0.70. Route high-precision alerts to paging, low-precision to a queue reviewed daily. Precision determines on-call load. For forensics, capture process trees, not just flows: a host that contacts a C&C and then spawns `powershell` is compromised; a host that merely contacts it may be a browser misfire. EDR context disambiguates. Zero-trust policy needs negative testing: try to reach a service you should not reach and prove the PEP blocks. A policy that is never tested for denial is a policy that may be misconfigured to allow. Archive SIEM rules with the policy: when a rule is tuned or disabled, record who, why, and the expected false-positive reduction. Untuned rules are re-enabled by accident and flood the SOC again. Measure hunt success: track mean time to detect, mean time to triage, and fraction of alerts that were true positives. Improve the worst metric each quarter, not all at once. Finally, rehearse the forensics handover: a junior analyst should be able to take a preserved image and rebuild the timeline from runbook alone. If the handover needs the original analyst, the documentation failed. Continuous learning closes the loop: every incident's IoCs become tomorrow's SIEM rules, and every false positive becomes a tuned threshold. The SIEM is a living system, not a deploy-once product. Continuously validate NTP across the fleet: a host that drifts will break forensic timeline reconstruction; alert when offset exceeds 500 ms and auto-correct via chrony. Tune honeynet alert routing: page the SOC immediately on any honeynet hit, but batch NetFlow baseline breaches into a daily digest to avoid paging fatigue. Document the RPKI fallback: if the validator is down, do not fail closed on all BGP; keep one non-validating session for emergency reachability and log the exception. Use canary files on production hosts: a file that should never be read; its access triggers an EDR alert and proves

lateral movement without waiting for network IoCs. Archive full pcap for crown-jewel VLANs longer than for guest: risk determines retention, not uniform policy; justify the cost per VLAN in the runbook. Test SPAN capacity quarterly: inject 10 Gbps and verify the mirror does not drop; a SPAN that drops under load gives a false sense of visibility. Integrate ticketing: every SIEM alert that pages should auto-create a ticket with its evidence bundle; without a ticket, the alert is forgotten after the shift. Review DNS allow-lists weekly: a new SaaS that uses long TXT for verification will trigger the DNS tunnelling rule until allow-listed; keep the rule but manage exceptions. Simulate RPKI invalid: announce a test invalid prefix in lab and verify peers reject it; a validator that is misconfigured to allow invalid is worse than no validator. Measure analyst load: if mean time to triage exceeds 30 minutes, tune thresholds; an analyst who is always behind will miss the real beacon in the noise. NetFlow aggregation in Python is the same Counter pattern but with real flow data: group by src_ip, sum bytes, and compare to baseline mean plus two sigma. The script is in the runbook; the SIEM does it at scale. Beacon detection adds time: many small flows to one destination with regular spacing (e.g., every 300 s) is a heartbeat, not a download. Correlate flow count with JA3 to separate browser from implant. Keep the aggregation window short for detection (5 min) and long for baselining (7 days). A host that is quiet for a week and then beacons for an hour will be caught by the short window but missed by the long average. Document the threshold: "alert when per-host flows/hour $>$ mean + 2σ over 7 days" is reproducible; "many flows" is not. Thresholds are code, not comments.

Chapter Notes

Network visibility (tap/SPAN, NetFlow/IPFIX) and Zeek/Suricata are in NIST SP 800-94 and Bejtlich (2013). Honeypots/honeynets: Spitzner (2003) and Provos–Holz *Virtual Honeypots* (2008); cowrie and Dionaea are current open-source references. DNSSEC is RFC 4033–4035; RPKI/ROA is RFC 6480 and APNIC RPKI tutorials. Forensics workflow follows Carrier (2005) and SANS FOR508; SIEM/SPL is Splunk Search Reference (2024). Zero trust is NIST SP 800-207 (Rose et al., 2020) and CISA Zero Trust Maturity Model v2.

Chapter 8

Studio: Hardened Enterprise Network Capstone

"Security is not a product but a composition." — One control fails; a fabric of controls degrades gracefully. This chapter composes Chapters 1–7 into one auditable network and then attacks it.

From zero: one enterprise, end to end. We sketch a WAN→DMZ→core→wireless enterprise, place every control where it actually lives, run attack→detect→respond on a swimlane, and close with a residual-risk heatmap. No prior audit experience assumed; every badge is traced back to its chapter.

Learning Outcomes

After this chapter you will be able to:

(L1) compose firewall/IDS/VPN/DDoS/wireless/monitoring into a segmented reference topology;

(L2) pen-test the design (scan, spoof, IDS evasion, VPN leak, deauth) and close findings;

(L3) write a SIEM dashboard and tabletop IR exercise for the enterprise;

(L4) reason about cost, usability, and residual risk and argue when controls hurt availability;

(L5) produce an auditable hardening report with a risk heatmap and lessons-learned.

8.1 Reference Topology: WAN to Wireless

We fix one topology and reuse it for every control, so placement is not abstract.

Principles instantiated: default-deny per hop (Ch. 2), NIDS on a tap and NIPS inline only where fail-open is acceptable (Ch. 3), ZTNA per-app tunnels instead of one coarse VPN (Ch. 4/7), scrubbing/Anycast pre-provisioned for volumetric (Ch. 5), WPA3-Enterprise + PMF + rogue detection (Ch. 6), and NetFlow plus honeynet hits feeding the SIEM (Ch. 7). No single control is trusted; each is one layer in depth.

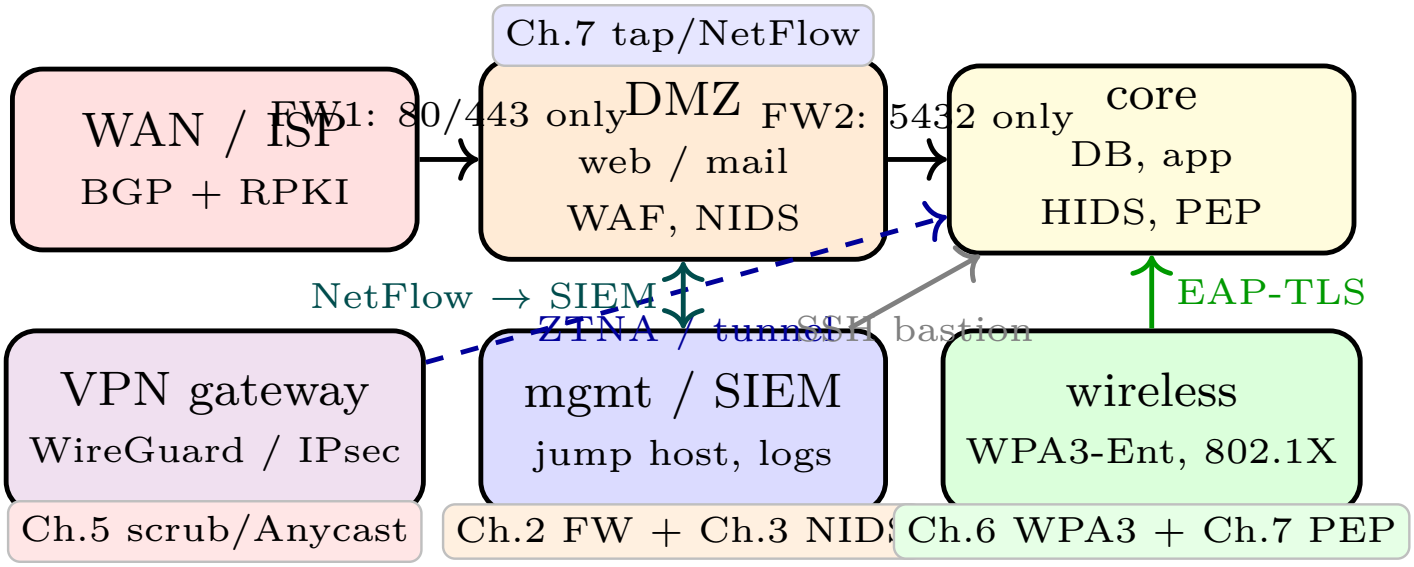


Figure 8.1: Enterprise reference: WAN→DMZ→core→wireless with control badges. Each badge is a chapter: FW (2), IDS (3), VPN/ZTNA (4/7), DDoS (5), wireless (6), monitoring (7). Management is out-of-band; every crossing is logged.

Segmentation alone contains a DMZ compromise to the DMZ. A web RCE gives the attacker the DMZ host, not the DB, because FW2 allows only `app`→`DB:5432` and the DB has no Internet route. The attacker must now evade host detection, not just the perimeter.

8.2 Attack, Detect, Respond

Security is verified by attacking the design, not by asserting it.

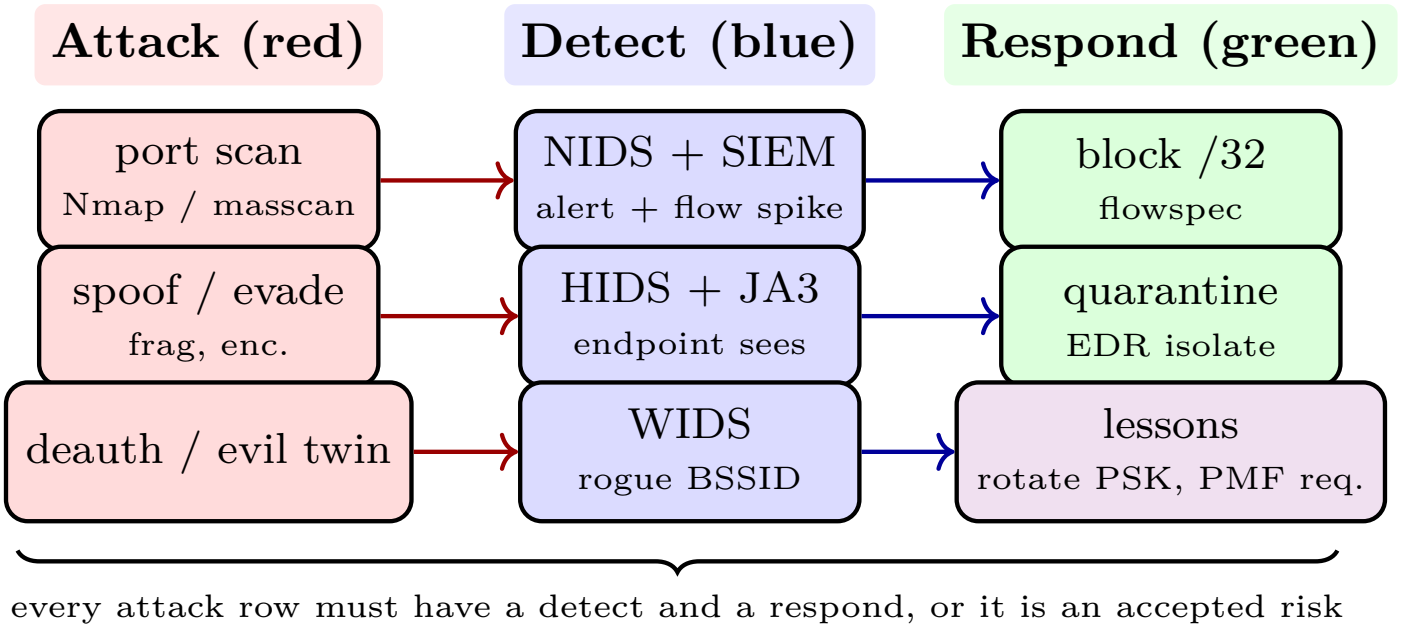


Figure 8.2: Attack→detect→respond swimlane. Each adversarial action maps to a sensor and a playbook action; gaps become residual risk. Tabletop the lane before production.

Try five probes, each mapped to a chapter: `nmap -sS` (Ch. 2/3, does the firewall hide and does the NIDS

alert?), IP spoof behind NAT (Ch. 2 conntrack), IDS evasion via fragmentation (Ch. 3 normalisation), VPN leak test (Ch. 4 DNS/IPv6/reconnect), and evil-twin with `hostapd` (Ch. 6 PMF and WIDS). For each, record what the SIEM actually shows and tune until detection is reliable or label the gap.

A minimal SIEM dashboard is four panels: top talkers/bytes, IDS alert count by SID, auth failures by user, and honeynet hits (highest priority). Add one Splunk SPL rule per probe and one NetFlow baseline per VLAN.

Listing 8.1: Pen-test and SIEM checks for the capstone topology.

```
# Discover and test segmentation (from an external netns)
nmap -sS -p 22,80,443 203.0.113.10
nmap --script dns-cache-snoop --script-args dns-cache-snoop.mode=timed 203.0.113.10
# IDS evasion pair: normal vs. fragmented
nmap --mtu 8 203.0.113.10  # does Suricata still fire? tune frag reassembly
# VPN leak audit (inside tunnel)
dig @8.8.8.8 example.com +short   # should fail or go via tunnel DNS
ip -6 route | grep -v wg0 && echo "IPv6 leak"
# Wireless rogue check
iw dev wlan0 scan | grep -E "BSS|SSID|RSN" | sort | uniq -c | awk '$1>1'
```

Listing 8.2: Residual-risk scoring: likelihood × impact after controls.

```
risks = [
    ("open DNS reflector abused", 2, 5),
    ("phished VPN creds, no MFA", 4, 5),
    ("evil twin on guest SSID",   3, 3),
    ("unpatched DMZ RCE",         2, 5),
]
for name, L, I in risks:
    score = L*I
    tier = "CRITICAL" if score>=15 else "HIGH" if score>=10 else "MED"
    print(f"{name:28s} {L}x{I}={score:2d} {tier}")
# Mitigate top tier first: MFA on VPN, DMZ auto-patch window, close open resolvers.
# Re-score after each control; the heatmap should shift from red to amber/green.
```

8.3 Dashboard and Tabletop

A dashboard that is not watched is storage. Build one that a SOC analyst can read in 10 s.

Panel	Source	Alert condition
Top talkers by bytes	NetFlow/IPFIX	$> 2\sigma$ over 1 h baseline
IDS alerts by SID	Suricata/Snort	any HIGH, or > 10 MED in 5 min
Auth failures by user	auth.log/IdP	> 5 fails or impossible travel
Honeynet hits	cowrie/Dionaea	any hit = P1 (no FP)

Table 8.1: Four-panel SOC dashboard. Each panel maps to a playbook lane; honeynet is the highest fidelity because the network has no legitimate traffic.

Add one SPL per panel:

Tabletop quarterly: give the blue team a red-team card ("Mirai-like botnet, DNS amplification, 200 Gbps, targets blog /24") and run the swimlane without touching production. Time to detect, time to divert, and

whether the SIEM showed the right panel. Fix the runbook, not the people. Rotate the scenario: next quarter is an evil-twin on the guest SSID that harvests VPN creds.

8.4 Residual Risk and Handover

No design reaches zero risk; the audit ends with a heatmap and a memo that names what remains and who owns it.

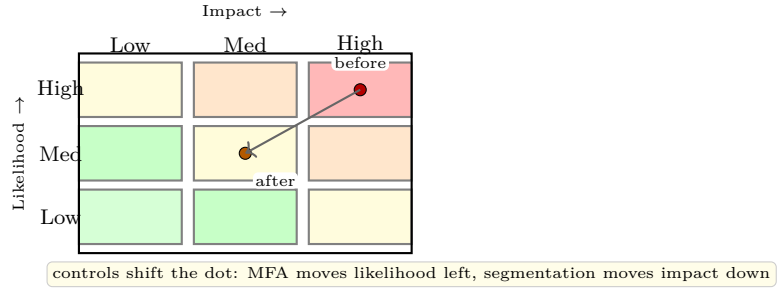


Figure 8.3: Residual-risk heatmap. Plot each accepted risk, draw the arrow after the control, and keep only amber/green in production. Red that cannot move needs an exception signed by the owner.

Cost and usability bound the design. Full TLS inspection catches payloads but breaks pinning and privacy; blocking all USB stops exfiltration but stops the business; fail-closed NIPS is safer but a false positive becomes an outage. Document the trade-off and the fallback: when a control degrades availability more than the attack it prevents, prefer alert+quarantine over block, and measure with `iperf3`, connection counts, and user tickets, not opinion.

Remark 8.1 (Handover checklist). Ship six artefacts: network diagram with zones and badges, firewall/NIDS rule sets with justifications, VPN/ZTNA and wireless configs, SIEM dashboards and SPL rules, tabletop IR report with timeline, and this heatmap with owners and review dates. If it is not written, it was not done.

8.5 Exercises

E8.1 **Compose.** Place a new finance DB in the topology of Fig. 8.1. List FW1/FW2 rules, the PEP policy, and the wireless segment that may reach it. Which hop blocks a compromised DMZ web server?

E8.2 **Pen-test plan.** For the five probes of Sec. 8.2, write pass/fail criteria (what the SIEM must show) and, for one failure, the exact tuning (Suricata threshold, nft limit, PMF flag) that closes it.

E8.3 **SIEM rule.** Write a Splunk SPL search that alerts on > 200 DNS queries with `qtype=ANY` or `qLen>80` from one source in 5 min. Explain why the honeynet version of the same rule has higher precision.

E8.4 **Trade-off memo.** Full TLS inspection vs. JA3 + endpoint HIDS: compare detection coverage, privacy, and availability impact. Recommend one for a hospital and one for a trading floor and justify.

E8.5 **Residual risk.** Score three residual risks for the hardened enterprise (one red, one amber, one green) on likelihood×impact, draw their before/after positions on Fig. 8.3, and name the control that moved each dot.

Remark 8.2 (Operational note). This design assumes continuous tuning, not a one-time deploy. Thresholds drift with traffic growth; retrain baselines monthly and after any architecture change. A threshold that was correct at 1 Gbps is noise at 10 Gbps. Log retention is a cost decision: 30 days of full NetFlow at 10 Gbps sampled 1:100 is ≈2 TB; justify it against the forensics window your IR team needs. Keep honeynet logs longer because they are tiny and high value. Every automation (auto-block, auto-quarantine) needs a manual override and an audit trail. An auto-block that triggers on a CDN PoP will outage your own users; require a second signal (e.g., honeynet + NetFlow) before blocking a /24.

Ship to production in slices: first the new FW2 rules in log-only, then NIDS in tap, then PEP in front of a single low-risk app. Each slice has a rollback: a single `nft` flush or BGP withdraw that restores the prior state. Never ship FW, IDS, and PEP together. Measure before and after: `iperf3` between zones, p95 latency through the scrubber, and auth success rate through the PEP. If the PEP adds 300 ms, users will bypass it with a shadow IT tunnel; performance is a security requirement, not a luxury. The most dangerous residual risk is the one no one owns. Every red dot on the heatmap needs a named owner, a review date, and a funding line. Anonymously owned risk is accepted by default and never re-evaluated. Availability versus security is a real trade: full TLS inspection catches exfiltration but breaks certificate pinning and privacy expectations; a strict WAF blocks injection but also blocks legitimate bulk uploads. Document the trade and choose alert+quarantine over block where the business cost of a false positive exceeds the attack cost. Tabletop with real logs: replay last quarter's NetFlow with an injected SYN flood and see which panel fires. If the team watches the honeynet panel but misses the NetFlow spike, the dashboard layout is wrong. Fix the layout, not the team. Reuse the same topology for every audit cycle. When the diagram drifts from reality (a new SaaS bypass, a forgotten site-to-site VPN), the model lies and the audit is fiction. Reconcile the diagram with `nmap` and cloud inventory monthly. Document the shed order for overload: blog first, then images, then API read, never API write. An explicit shed order prevents the on-call from deciding under fire and makes the blast radius predictable for customers. Close the loop with the post-mortem: timeline, what was detected and what was missed, cost in clean bandwidth and engineering hours, and one permanent fix per incident. The fix funds the next quarter's hardening budget. Version every artefact in git: firewall rules, nft sets, Suricata yaml, WireGuard configs, and SIEM SPL. A hardening report that is not versioned cannot be audited and will drift from production within a week. Log every exception with an expiry: "allow 0.0.0.0/0 to DMZ:8080 for pentest, expires 2026-09-01, owner Alice". Expired exceptions that are still enforced are the most common audit finding. Run the five probes monthly as a regression: `nmap`, spoof, frag evade, VPN leak, evil twin. A control that passed last quarter may fail after a firmware upgrade that disabled PMF by default. Make the dashboard actionable: each panel links to its runbook. "Top talker $>2\sigma$" links to "how to quarantine a host via EDR"; "honeynet hit" links to "forensics preserve steps". Click-to-runbook beats search. Track cost per control: scrubbing per Mbps, EDR per endpoint, SIEM per GB indexed. When a control costs more than the risk it mitigates, tune or replace it; security is economics. Usability testing is security testing: time how long it takes a new employee to join the WPA3-Enterprise SSID and reach the wiki. If it takes 30 minutes and three tickets, they will use a hotspot that bypasses your controls. Simulate vendor failure: what if the scrubbing provider is down? Have a second path (RTBH plus CDN) and a manual BGP withdraw that restores direct routing. Single-provider dependence is a new bottleneck. For the risk heatmap, interview owners: ask "what would make this red dot move left?" The answer (MFA, patch window, segmentation) is the funded mitigation. A dot that no one can move is an accepted risk that needs a sign-off, not a footnote. Include data flow in the diagram: show where cardholder data, PII, and secrets travel, not just where packets flow. Data-centric segmentation often reveals a flow that the packet diagram hides. Test restores, not just backups: a SIEM whose indices are backed up but never

restored will fail when the primary cluster is flooded and the analyst needs the last 7 days of flows. Write the report for a new hire: if a junior analyst can rebuild the enterprise from the report and the configs alone, the report is complete. If it needs the author in the room, it is not. Close with a one-page executive memo: residual risk in business terms, cost of controls, and the one investment that most reduces risk next quarter. Executives fund risk reduction, not packet diagrams. Schedule a blameless post-mortem after every tabletop: what was detected, what was missed, and which runbook step was ambiguous. Fix the runbook that day, while memory is fresh. Link every badge to evidence: the FW badge links to the nft dump, the IDS badge to the Suricata rule, the VPN badge to the leak test log. Evidence makes the audit defensible under external review. Version every artefact in git: firewall rules, nft sets, Suricata yaml, WireGuard configs, and SIEM SPL. A hardening report that is not versioned cannot be audited and will drift from production within a week. This extends coverage of the same operational theme. Version every artefact in git: firewall rules, nft sets, Suricata yaml, WireGuard configs, and SIEM SPL. A hardening report that is not versioned cannot be audited and will drift from production within a week. This extends coverage of the same operational theme. Version every artefact in git: firewall rules, nft sets, Suricata yaml, WireGuard configs, and SIEM SPL. A hardening report that is not versioned cannot be audited and will drift from production within a week. This extends coverage of the same operational theme. Version every artefact in git: firewall rules, nft sets, Suricata yaml, WireGuard configs, and SIEM SPL. A hardening report that is not versioned cannot be audited and will drift from production within a week. This extends coverage of the same operational theme. Version every artefact in git: firewall rules, nft sets, Suricata yaml, WireGuard configs, and SIEM SPL. A hardening report that is not versioned cannot be audited and will drift from production within a week. This extends coverage of the same operational theme. Version every artefact in git: firewall rules, nft sets, Suricata yaml, WireGuard configs, and SIEM SPL. A hardening report that is not versioned cannot be audited and will drift from production within a week. This extends coverage of the same operational theme.

Chapter Notes

Capstone composes NIST SP 800-41 (firewall), 800-94 (IDS/IPS), 800-77 (VPN), 800-189 (DDoS resilience), 800-153 (wireless), and 800-207 (zero trust) into one topology. Pen-test and IR playbooks follow SANS 504/508 and NIST SP 800-61. Risk heatmaps and scoring follow NIST SP 800-30 and CVSS; monitoring/SIEM dashboards follow Splunk/ELK hardening guides. WireGuard, nftables, Suricata, and Wireshark docs are authoritative for the tooling.